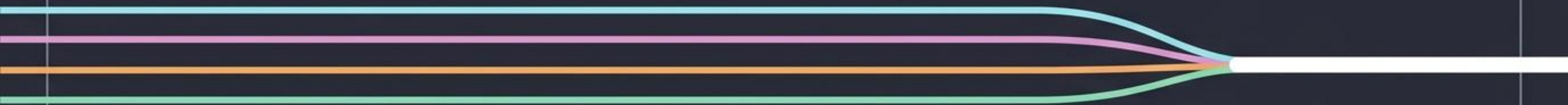




웹 애플리케이션의 해부학

표면(UI)에서 심연(DB)까지: 풀스택 개발을 위한 시각적 레퍼런스 가이드



웹 개발의 기초: HTML & CSS 핵심 요약 가이드

웹 페이지의 구조를 설계하는 HTML 태그와 디자인을 입히는 CSS의 핵심 요소를 한눈에 파악하기 위함.

HTML - 웹 페이지의 구조와 요소

블록(Block) vs 인라인(Inline) 요소

```
<div class="block">블록 요소 (화면 전체)</div>
<p class="block">블록 요소 (화면 전체)</p>
```

화면 전체를 차지하는 div, p(블록)

```
<span class="inline">인라인 요소</span>
필요한 공간만 차지하는 드롭 요소 이직을 위한
싱징으로 맥싱 징잉 척듬하는 메어 진초를 적응합니다.
<span class="inline">인라인 요소</span>
필요한 공간만 차지하는 트핑 요소 이화를 여지한
일적으로 확이 하는 소소의 일라인 요소를 적용합니다.
<a class="inline" href="#">링크</a>
```

필요한 공간만 차지하는 span, a(인라인)를
구분하여 배치합니다.

데이터 전송을 위한 폼(Form) 구성

```
<form action="/submit">
```

```
<input type="text">
```

이름 입력

```
<input type="password">
```

.....

```
<input type="checkbox">
```

동작 확인

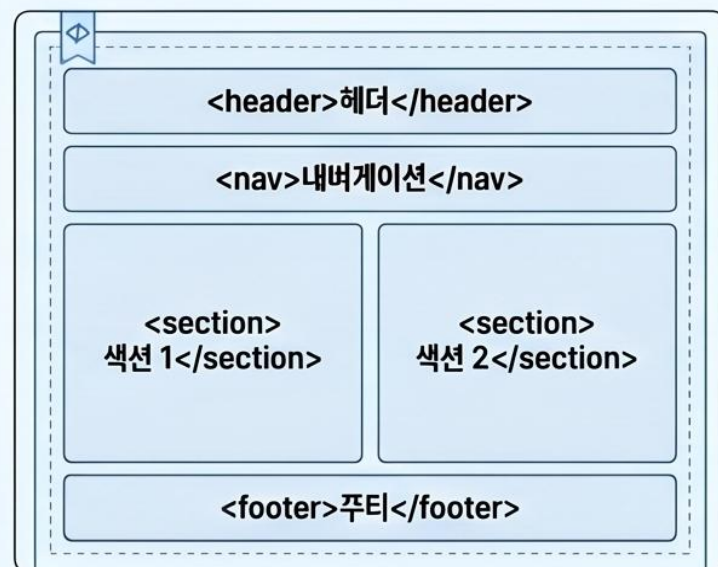
...

```
<button>전송</button>
```



input 태그의 다양한 타입(text, password, checkbox)를 form 태그로 감싸 서버로 데이터를 전달합니다.

시멘틱(Semantic) 레이아웃 태그



header, nav, section, footer 등을 사용하여
웹사이트의 구조를 의미론적으로 분할합니다.

CSS - 스타일링과 레이아웃 설계

핵심 선택자(Selector) 활용

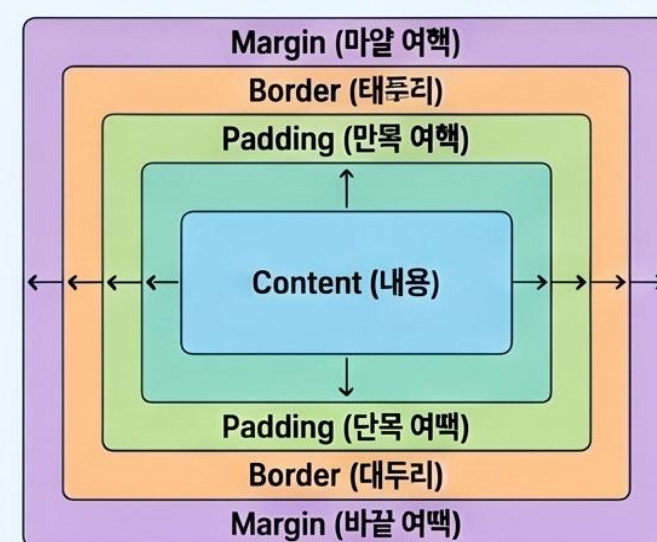


전체(*), 태그, 아이디(#), 클래스(.) 선택자를 통해
특정 요소에만 디자인을 적용합니다.

박스 모델(Box Model) 이해

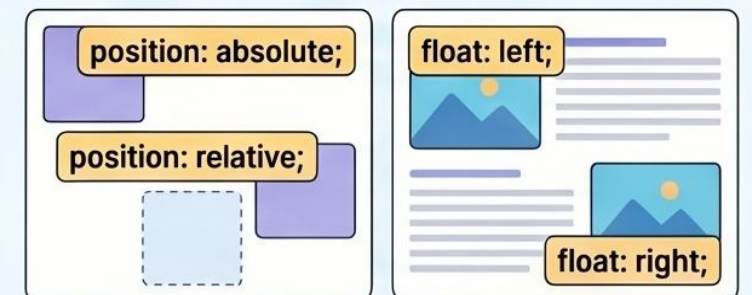


박스 모델(Box Model)의 이해



테두리(Border)를 기준으로 바깥 여백은 Margin,
안쪽 여백은 Padding으로 조절합니다.

레이아웃 배치를 위한 Position & Float

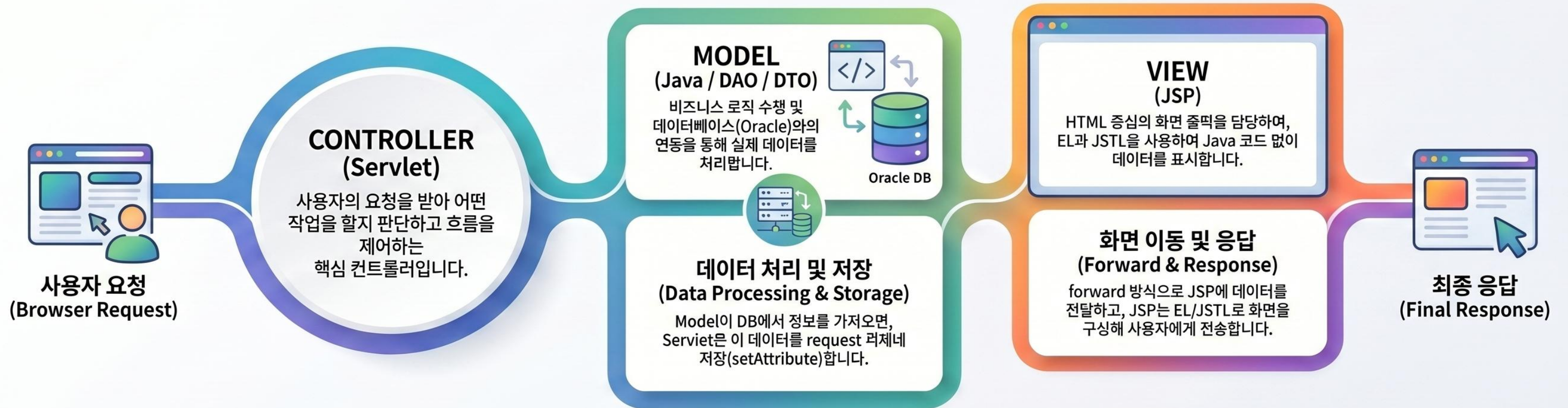


absolute, relative 등 위치 속성과 float를 이용해
요소를 원하는 곳에 정형합니다.

CSS 박스 모델 속성 및 정렬 방식 요약

속성 분류	주요 속성 및 값	설명
박스 속성	margin: 0 auto;	요소를 화면 중앙에 정렬
가시 속성	display: none;	요소를 화면에서 숨김 처리
위치 속성	z-index: [숫자];	숫자가 높을수록 네이어를 상단에 배치

JSP MVC 웹 애플리케이션 구조와 흐름 한눈에 보기



개발 시 혼동하기 쉬운
서버 내부 이동(Forward)과 재요청(Redirect)의 차이점

구분	forward	redirect
이동 방식	서버 내부에서 이동	브라우저가 다시 요청
데이터 유지	request 객체 유지 (O) ✓	request 데이터 사라짐 (X) ✗
URL 변경	변경되지 않음	새로운 URL로 변경

JavaScript 입문자를 위한 기초 핵심 요약 가이드

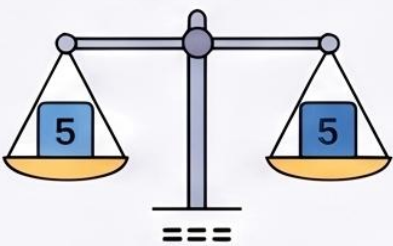
JavaScript는 웹 페이지에 동작인 움직임을 부여하는 프로그래밍 언어입니다.
본 가이드는 변수 선언 방식부터 데이터 타입의 종류, 필수 연산 법칙, 그리고 프로그램의 논리를 구성하는 제어문과 배열 관리법까지의 필수 기초를 다룹니다.

현대적 문법과 데이터의 기초

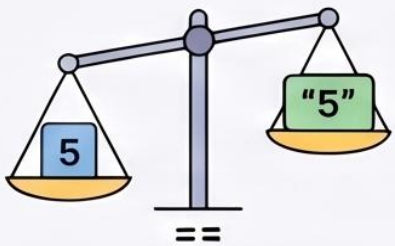
변수 선언: let과 const 사용 권장

var	let	const
 옛날 방식 (재선언 가능)	 현재 표준 (지역 변수)	 상수형 변수

엄격한 비교 연산자 '===' 활용

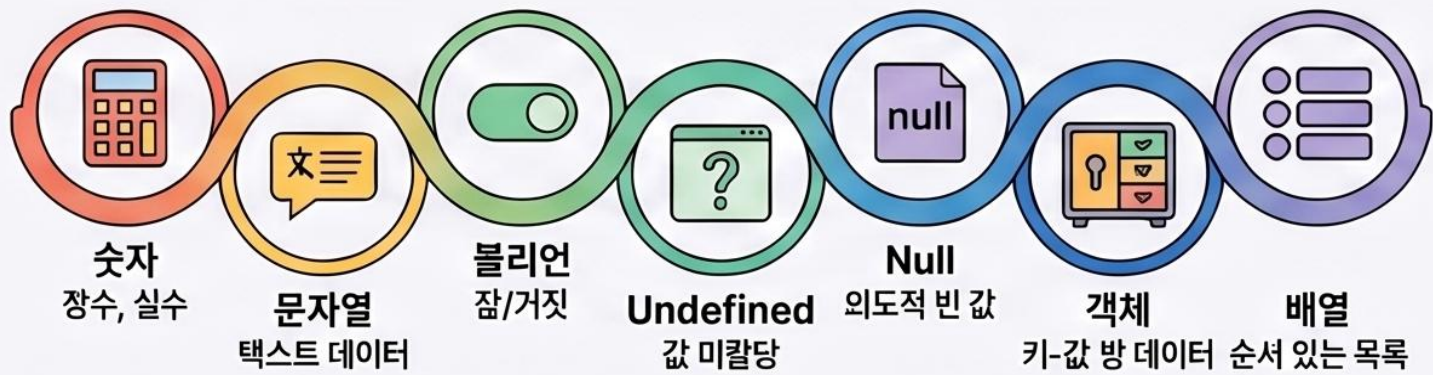


값과 자료형 모두 비교 (실루 표준)



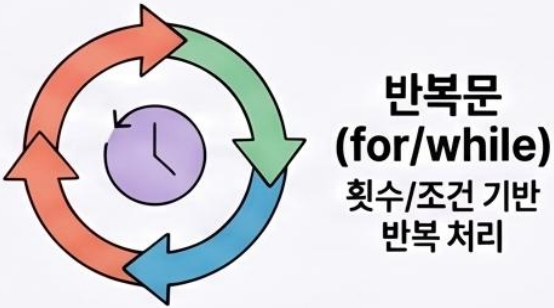
값만 비교 (예기치 많은 결과 발생 가능)

7가지 핵심 자료형

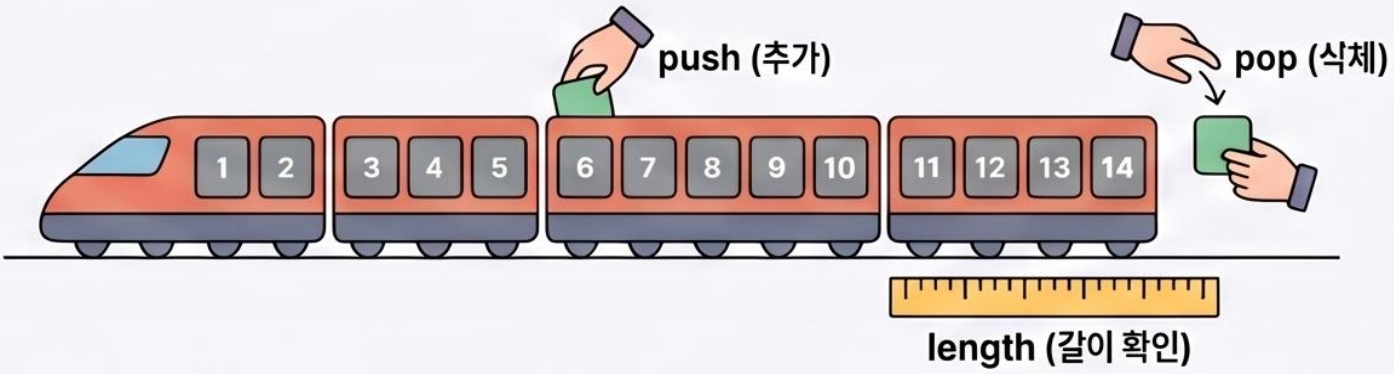


로직 제어와 데이터 구조

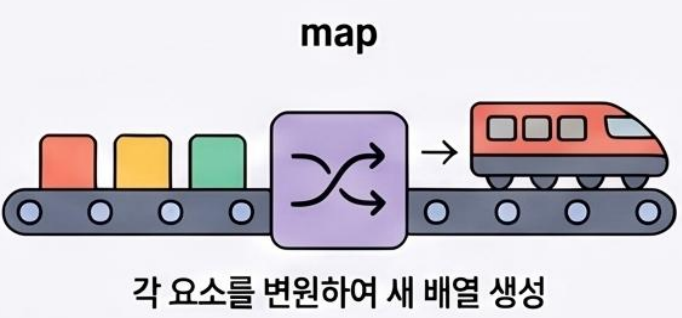
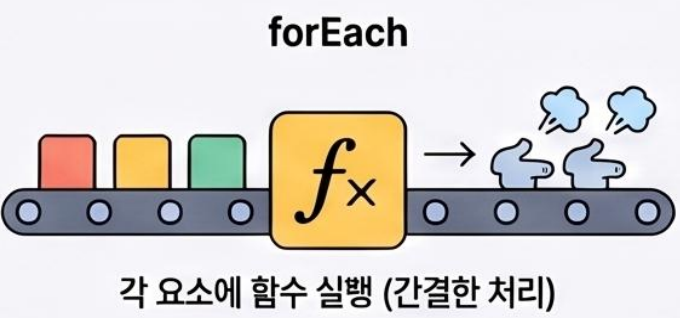
흐름 제어: 조건문과 반복문



데이터 관리의 핵심, 배열(Array)



효율적인 배열 반복 'forEach & map'

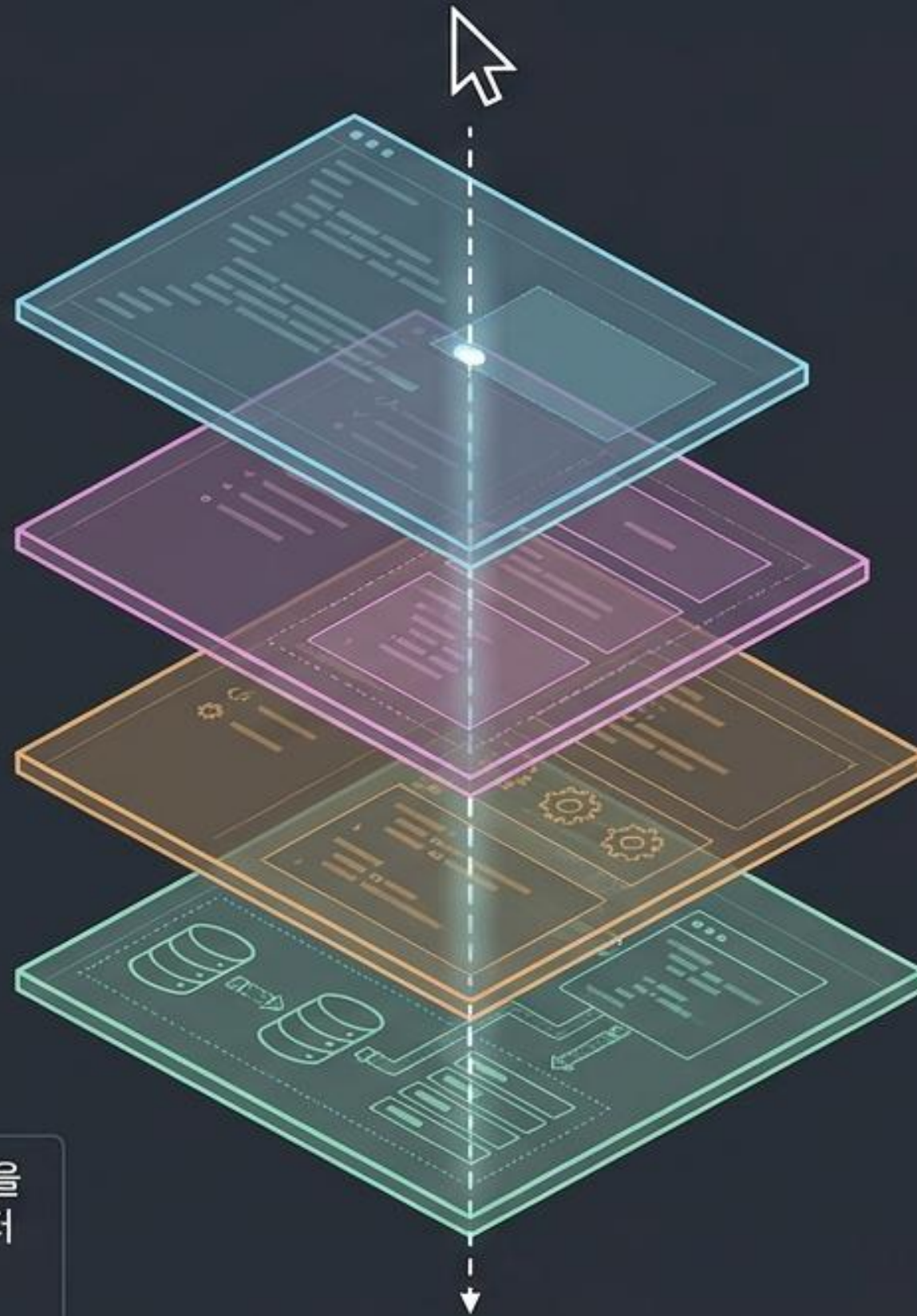


HTML (Markup Language)
데이터와 콘텐츠의 뼈대

CSS (Design & Layout)
시각적 생명력과 공간 배치

JSP (Server-side Rendering)
동적 데이터 처리 및 웹 서버 엔진

JDBC & DB (Data Core)
영구적인 데이터 보관 및 파이프라인



브라우저에서 시작된 단 한 번의 클릭은 이 4개의 지층을 관통하며 데이터를 변환합니다. 가장 바깥쪽의 뼈대부터 가장 깊은 곳의 데이터베이스까지 점진적으로 하강해 봅니다.

텍스트 (Text)

`<h1>~<h6>` : 제목 (Block + Bold)

`<p>` : 단락 / `<br>` : 줄바꿈

`<strong>`, `<b>` : 강조

`<pre>` : 편집된 내용 그대로 출력

멀티미디어 & 링크 (Media & Link)

`<img>` : src, width/height, alt

`<embed>` : 동영상 출력 /

`<iframe>` : 다른 HTML 추가

`<a>` : 화면 이동 (href, target)

목록 & 표 (List & Table)

`<ul>` > `<li>` : 리스트 형태 데이터

`<table>` > `<tr>` > `<th>/<td>` : 격자형 데이터

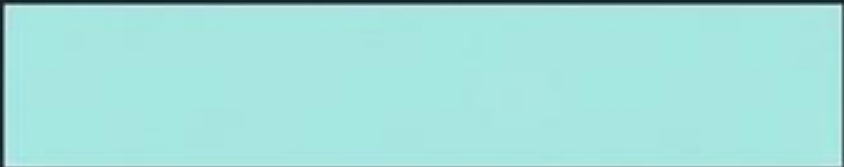
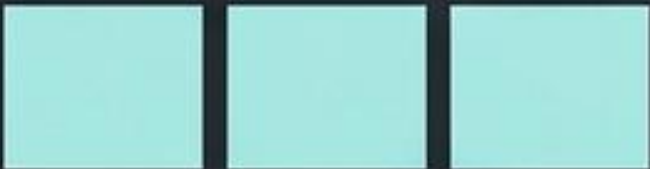
입력 폼 (Forms)

`<form>` : action, method

`<input>` : type = text, password, radio, checkbox, file, hidden

`<select>` / `<textarea>` / `<button>`

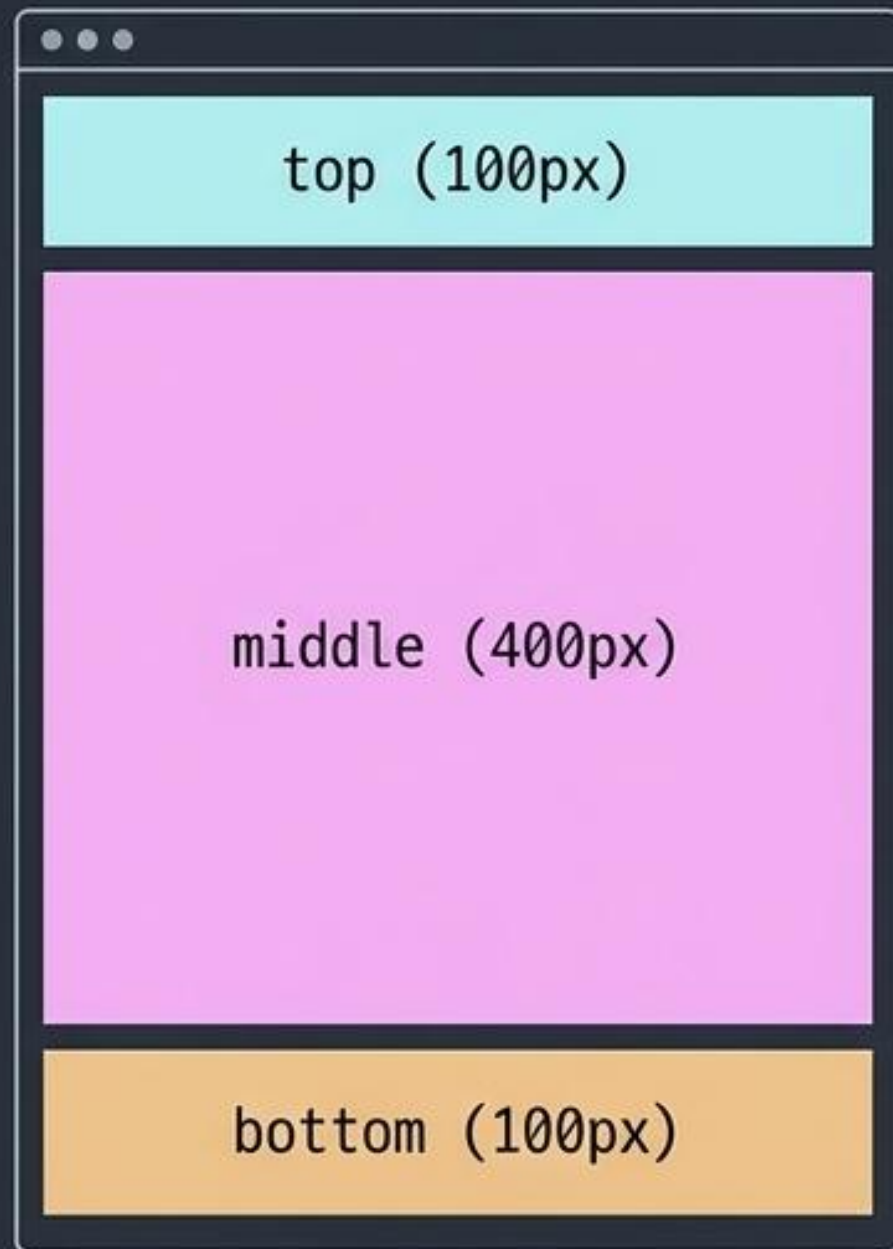
공간 점유 방식

Block 요소	 한 줄 전체 차지, 자동 줄바꿈. 예: <code><div></code>, <code><p></code>, <code><h1></code>, <code></code>, <code><table></code>
Inline 요소	 콘텐츠만큼만 차지, 옆으로 나열. 예: <code></code>, <code><a></code>, <code></code>, <code><input></code>, <code></code>

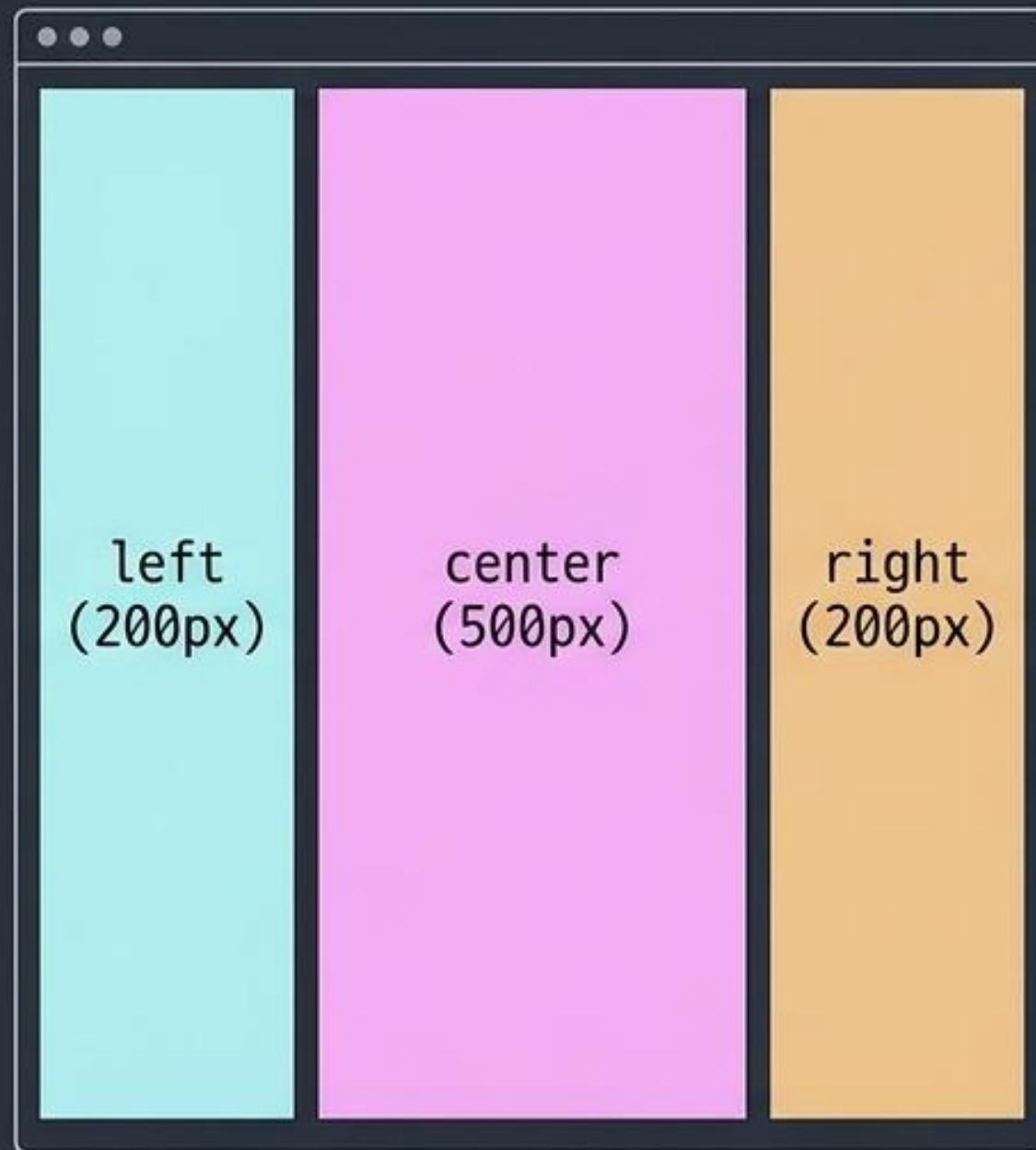
시맨틱 웹 구조



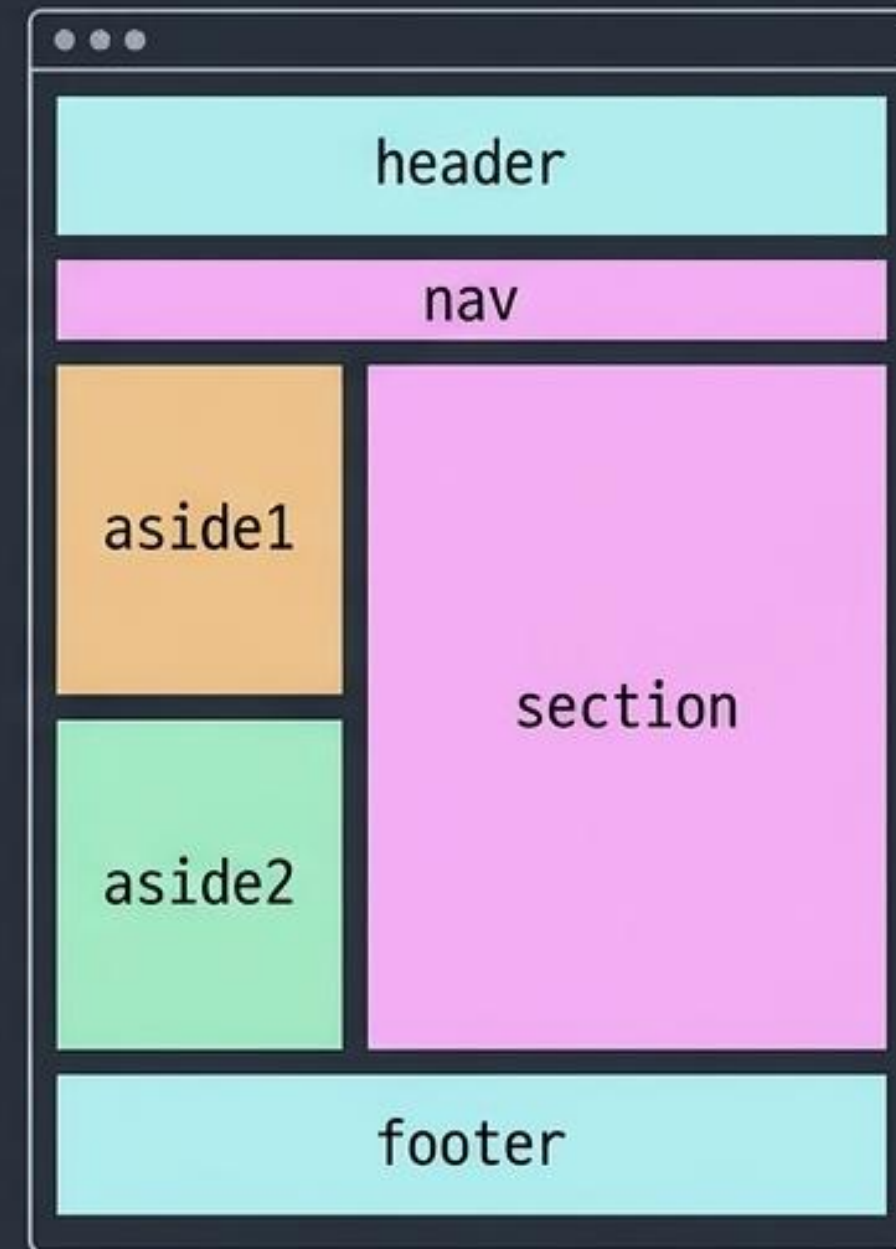
상중하 구조



3단 컬럼 구조



복합 레이아웃



GET (조회/노출형)



- URL 형태: ?변수=값&변수=값...
(URL에 데이터 노출)
- 용도: 페이지 이동, 단순 검색어, 상세보기
- 특징: 전송하는 데이터 크기에 제한이 있음

POST (전송/밀봉형)



- URL 형태: URL에 노출되지 않음
(HTTP Body에 숨김)
- 용도: 회원가입, 비밀번호 전송,
대용량 데이터 업로드
- 특징: `<form>` 전송을 통해서만 가능.
보안에 유리

가상 & 속성 선택자

상태:

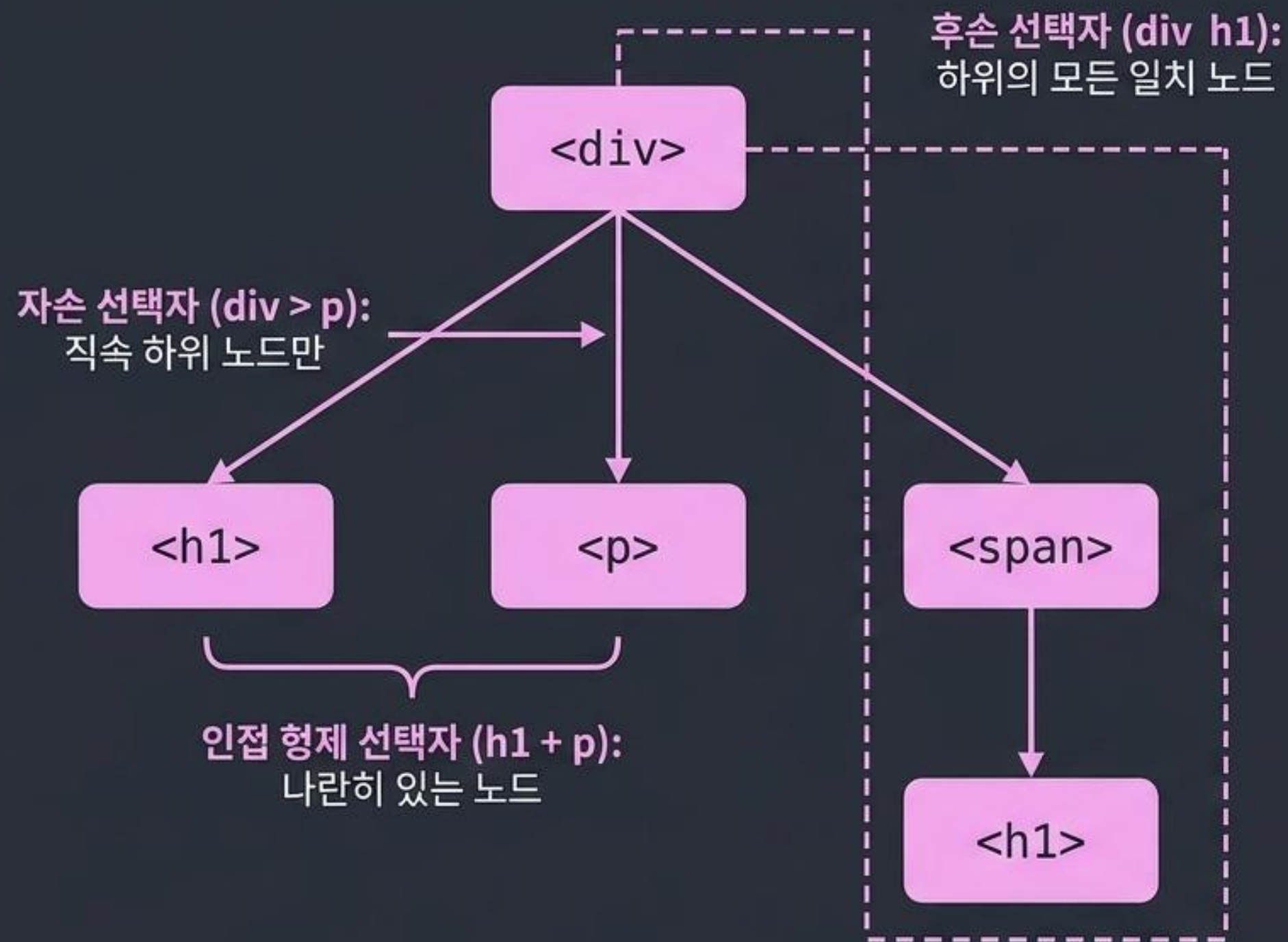
:hover,
:focus, :checked

구조:

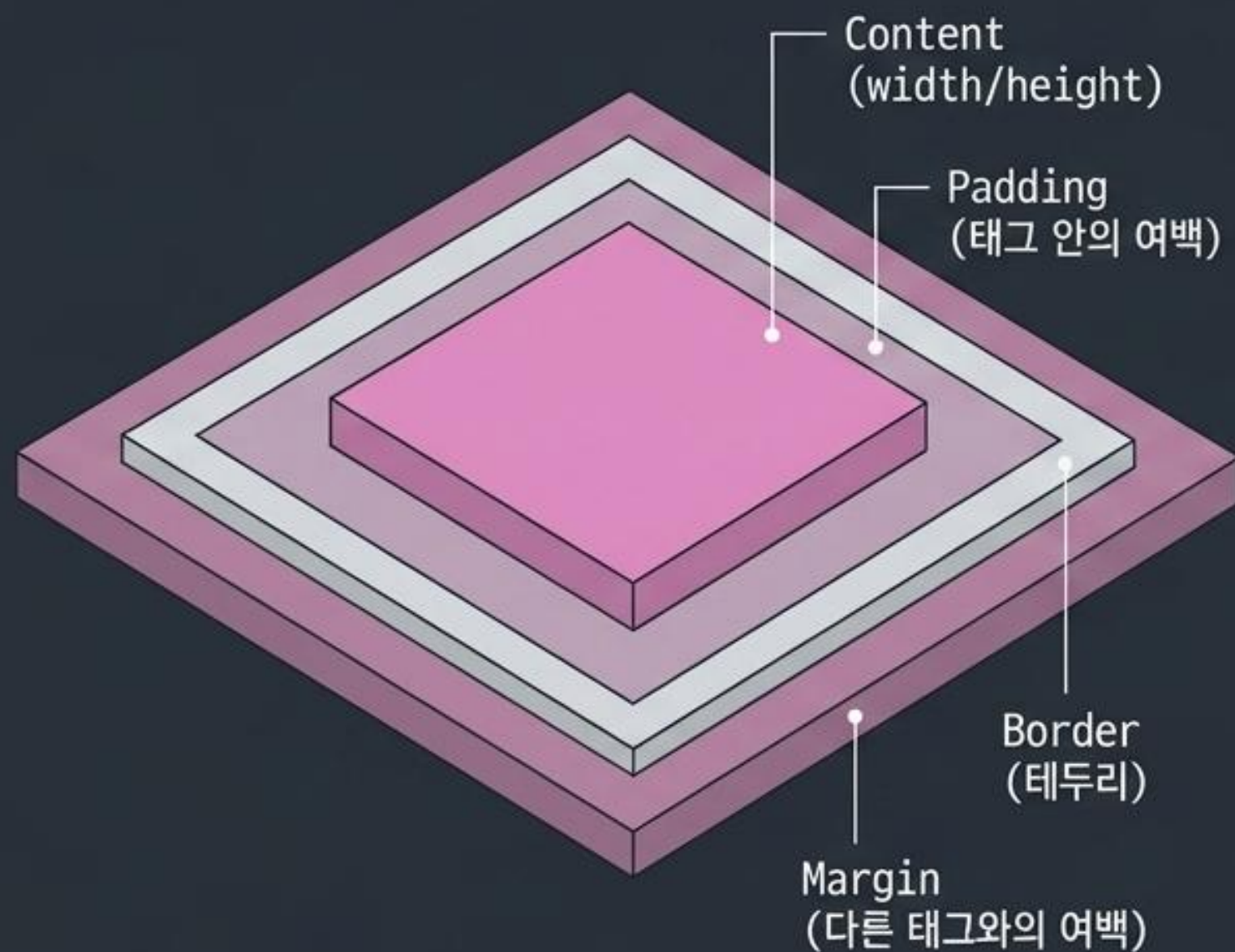
:first-child,
:nth-child(even/odd)

속성:

[속성=값],
[속성=값],
[속성^=값],
[속성\$=값]

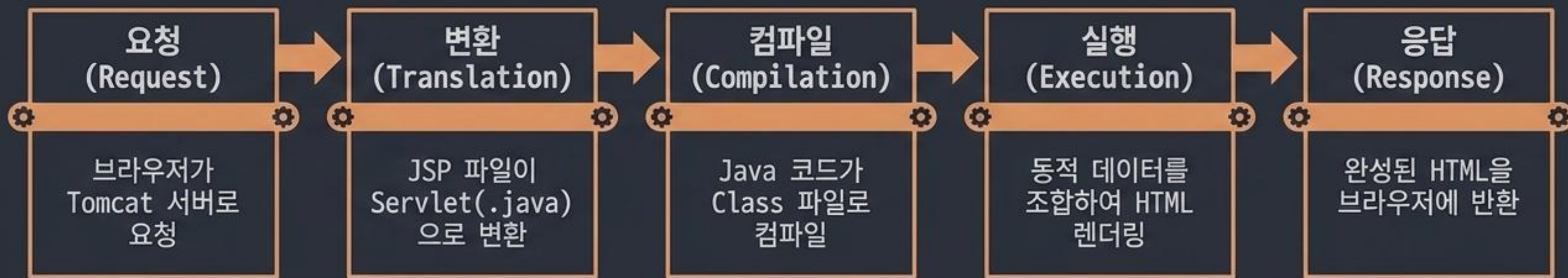


CSS 박스 모델 (Box Model)



배치와 좌표계 (Position)

- **static**: 코딩 순서대로 (기본값)
- **relative**: 원래 자기 위치를 기준으로 상대적 이동
- **absolute**: 부모/브라우저를 기준으로 절대 좌표 이동
- **fixed**: 화면 스크롤과 무관하게 고정 위치
- **float**: 좌/우로 띄우기 (해제는 `clear:both`)
- **z-index**: 높을수록 화면의 가장 위 레이어로 겹침



핵심 포인트: JSP는 런타임에 항상 Servlet으로 변환되어 실행된다.

JSP 지시자 (Directives)



page : 문서 속성 설정
(language, contentType, import)



include : 외부 파일 정적 포함
(`<%@ include %>`)



taglib : JSTL 등 확장 태그 사용

JSP 핵심 내장 객체



Active

request : 사용자의 요청 정보 보관



Ready

response : 응답 정보 통제



Active

session : 개별 사용자의 상태 유지
(로그인 등)



Active

application : 서버 전체의 공통 정보



Ready

out : HTML 화면으로 직접 출력



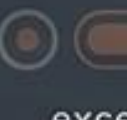
Active

pageContext



Ready

config



Ready

exception



Ready

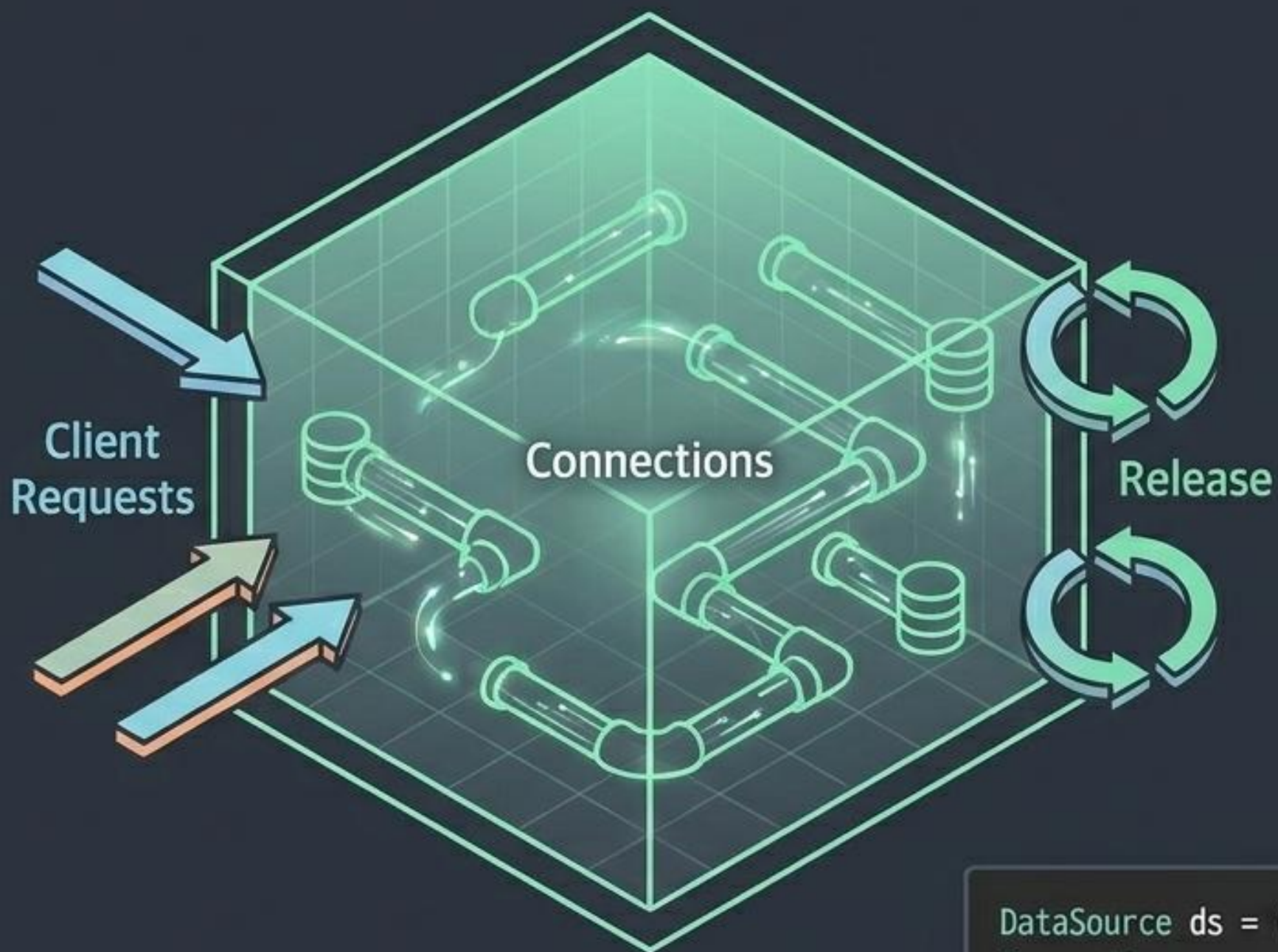
page

- 
1. Driver Load : DB 드라이버 클래스 메모리 적재
 2. Connection : DB와 물리적 연결 통로 개통 (getConnection)
 3. SQL 작성 : 실행할 쿼리문 준비
 4. Statement 생성 : SQL을 나를 운반 객체 생성 (prepareStatement)
 5. SQL 실행 : DB에 쿼리 전송 (executeQuery)
 6. ResultSet 처리 : 받아온 테이블 결과값(rs.next) 읽기
 7. 자원 반납 (Close) : 메모리 누수 방지를 위한 역순 종료 (rs, pstmt, conn)

비교 항목	Statement	PreparedStatement
SQL 완성 시점	실행 시점에 조립	미리 컴파일됨
처리 속도	느림	빠름 (캐싱 활용)
가독성	문자열 더하기(+)로 복잡	? 바인딩으로 깔끔함
보안성	SQL Injection 공격에 무방비	특수문자 이스케이프 처리로 안전



현업 표준: 보안과 성능을 위해
무조건 **PreparedStatement** 사용 권장



Connection Pool (CP) 개념

미리 여러 개의 Connection 객체를 생성해 수조(Pool)에 보관해 두고, 요청 시 대여 및 사용 후 반납하는 순환 시스템.

도입 효과

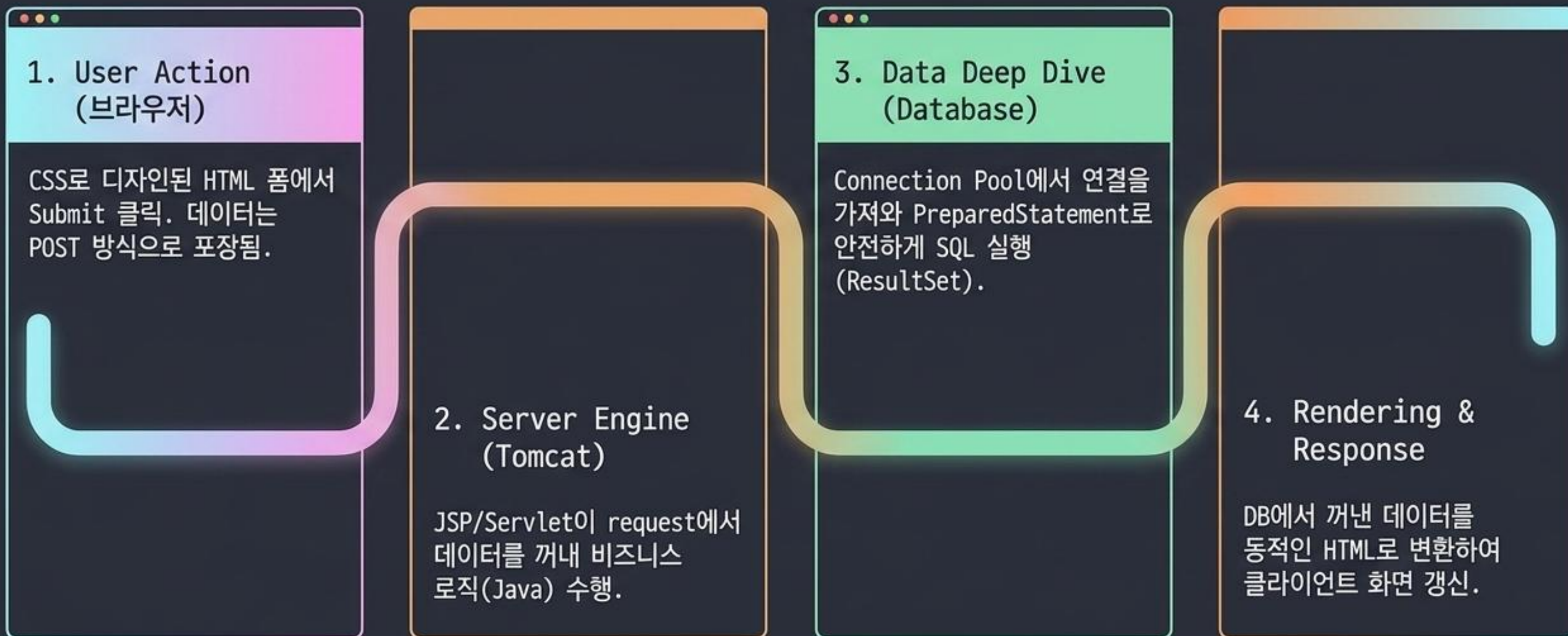
속도 극대화 (연결 시간 단축), DB 과부하 방지 및 서버 리소스의 안정적 관리.

대표 라이브러리 및 사용

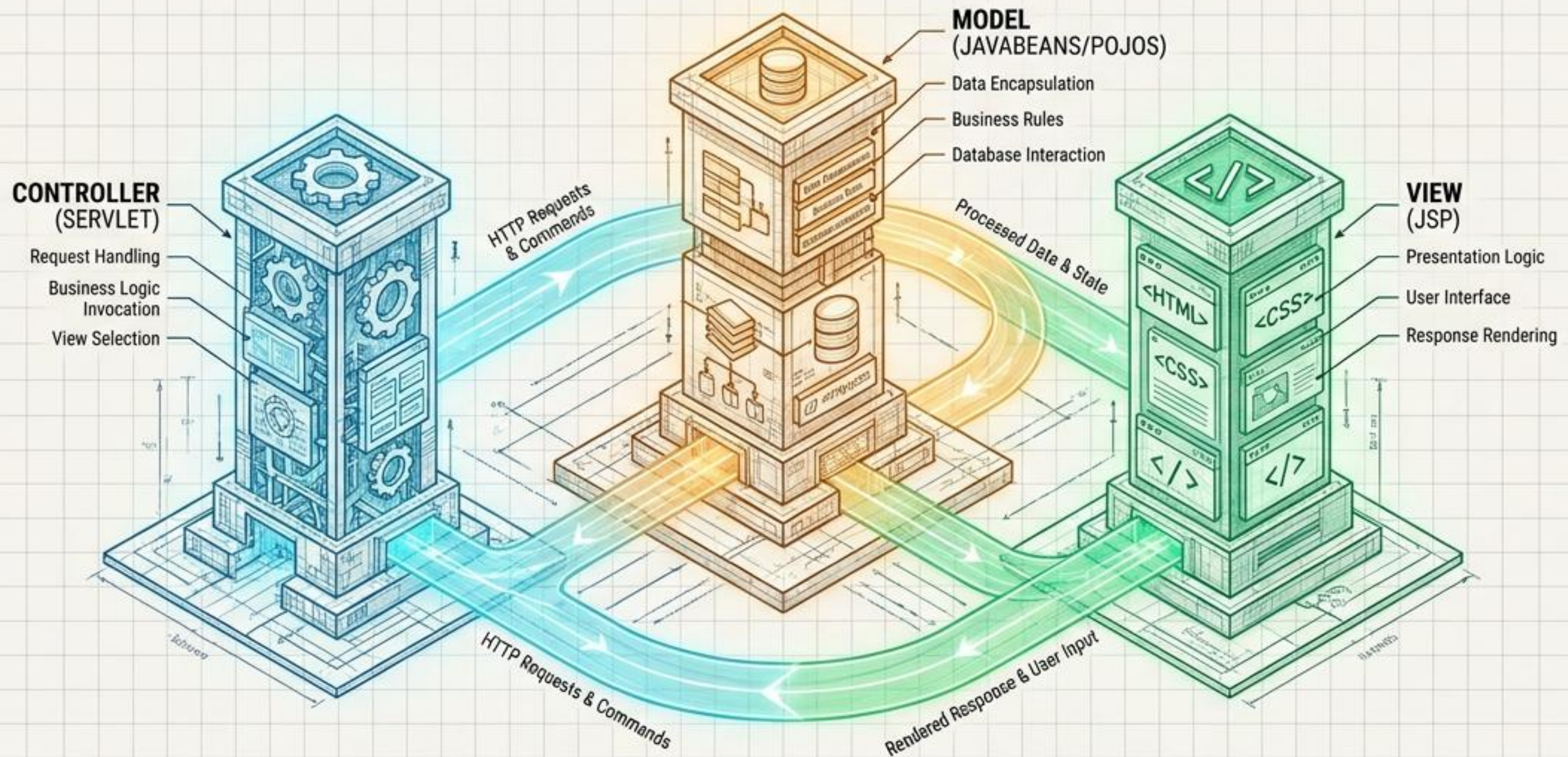
DBCP, HikariCP (현업 표준), C3P0

```
DataSource ds = (DataSource)init.lookup("java:comp/env/jdbc/oracle");  
Connection conn = ds.getConnection();
```

The Full-Stack Request Lifecycle



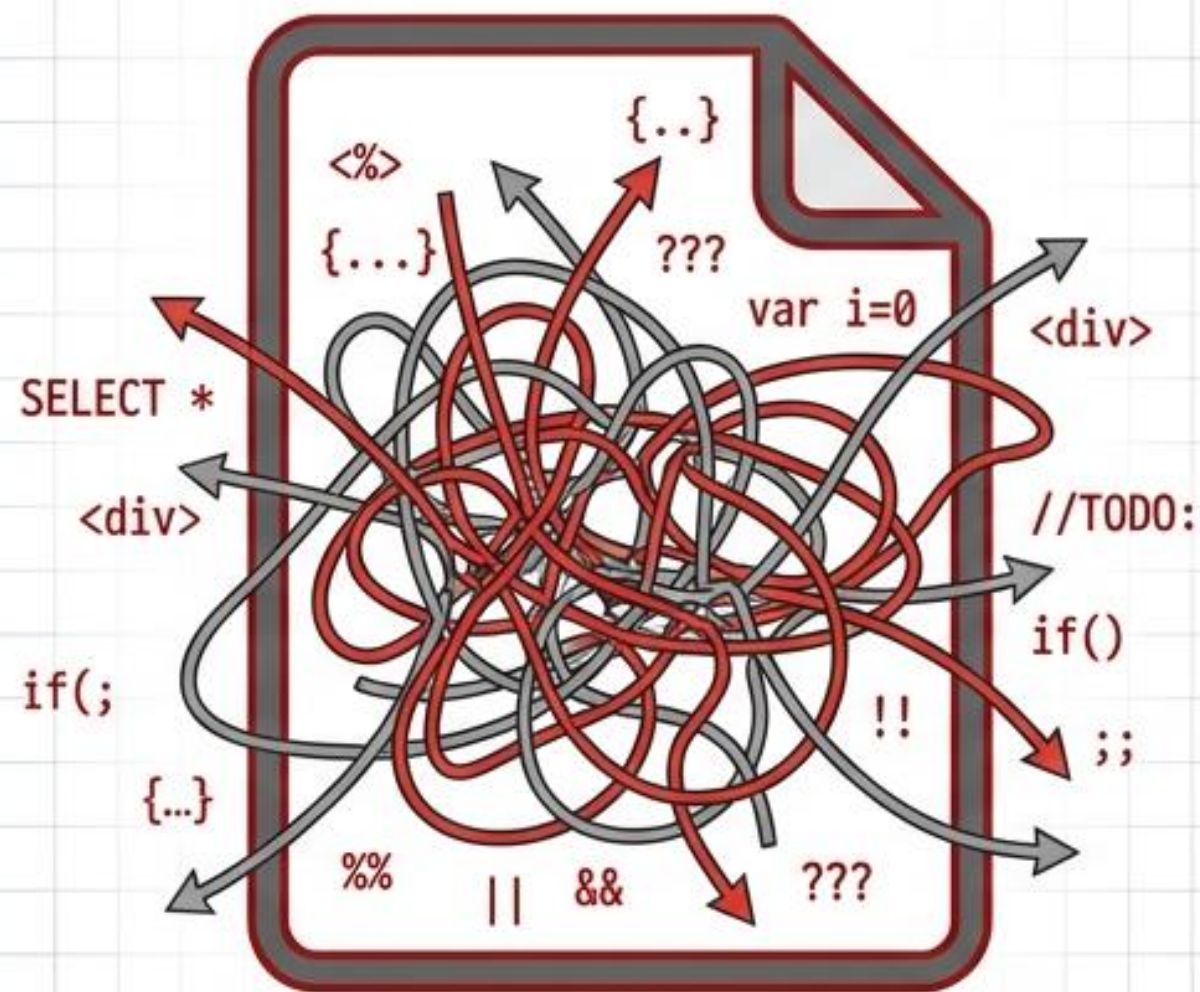
이 복잡하고 정교한 파이프라인의 이해가 풀스택 개발자의 진정한 시작점입니다.



웹 애플리케이션의 정석: JSP MVC 아키텍처 완벽 해부

유지보수와 확장성을 극대화하는 웹 백엔드 설계의 청사진

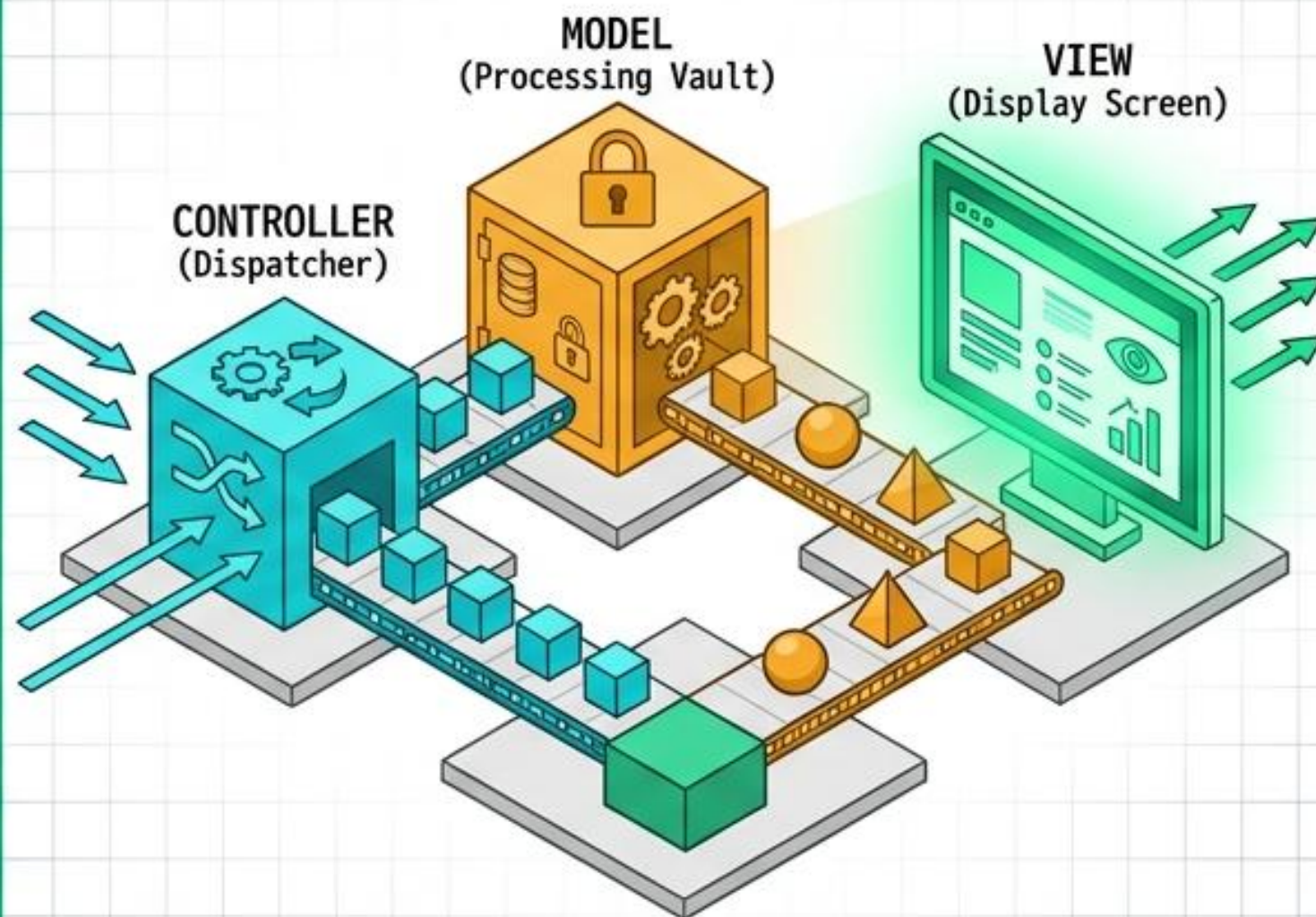
MVC1: SPAGHETTI CODE CHAOS



MVC1: 스파게티 코드

모든 로직(HTML, Java, DB)이 하나의 JSP 파일에 혼재.
유지보수 지옥 발생 및 작업 충돌.

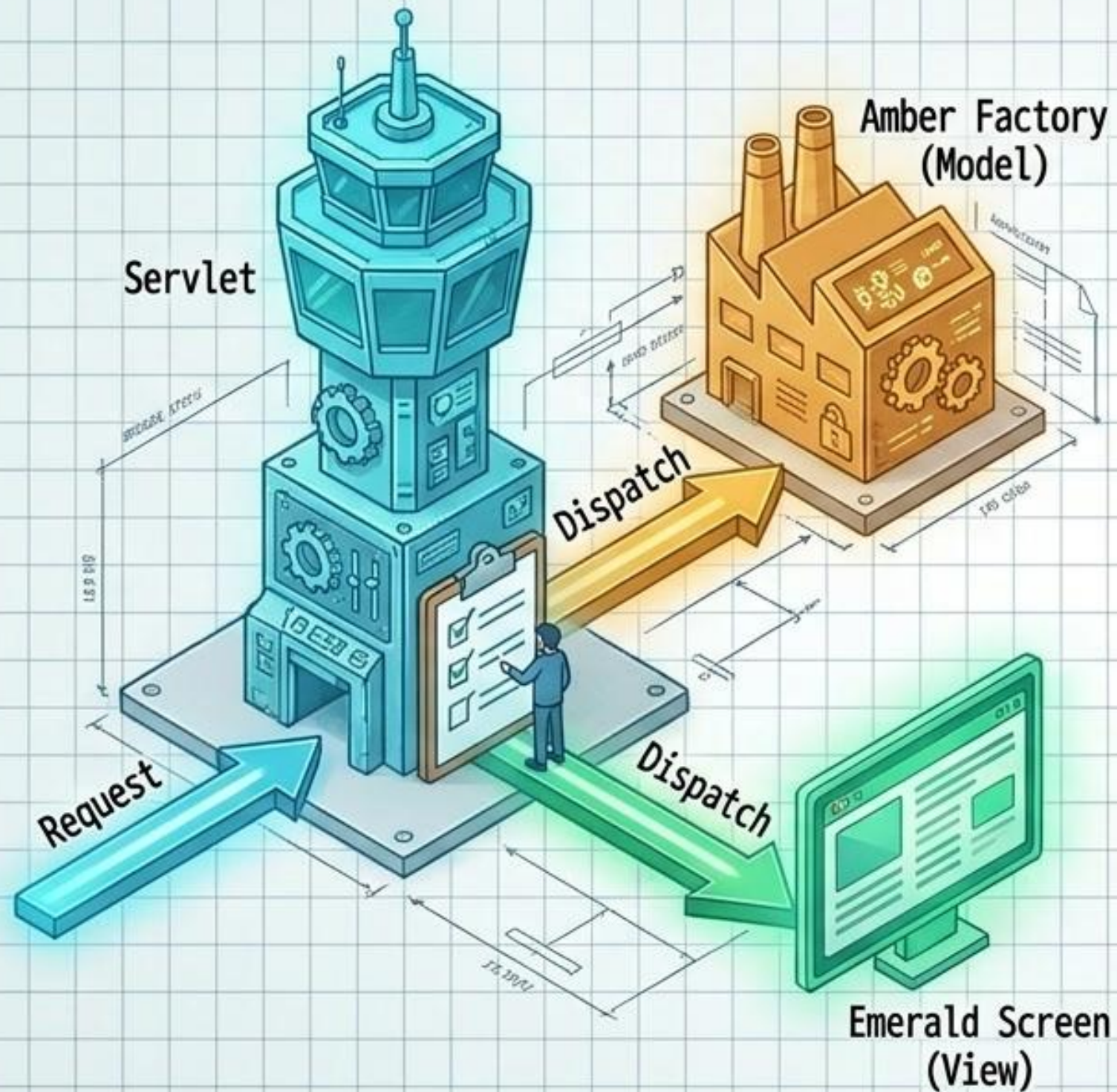
MVC2: STRUCTURED DIVISION OF LABOR



MVC2: 체계적인 분업화

Controller, Model, View로 역할을 명확히 분리.
현재 웹 개발의 표준 방식 및 코드 재사용성 극대화.

HTTPSERVLET: HANDLING REQUESTS & ROUTING



Servlet Syntax Overview

```
public class MemberServlet extends HttpServlet {  
    protected void doGet(...) { }  
    protected void doPost(...) { }  
}
```

Annotations in the original image:
- 'Inherits Servlet functionality' points to 'extends HttpServlet'
- 'Handles GET requests' points to 'doGet(...)'
- 'Handles POST requests' points to 'doPost(...)'

GET vs. POST: Key Differences

	doGet() (GET 방식)	doPost() (POST 방식)
데이터 노출	✓ O (URL에 표시)	✗ X (Body에 숨김)
길이 제한	있음	거의 없음
주 사용처	조회 / 검색	로그인 / 회원가입
특징	속도 빠름	보안성 상대적 우수

Forward (서버 내부 이동)

http://server.com/app/servletA



Request

Servlet A

request

JSP B

서버 내부 이동. 속도 빠름.
request 데이터 유지. URL 변경 없음.

```
rd.forward(request, response);
```

Redirect (브라우저 재요청)

http://server.com/app/jspB



Request 1

Servlet A

request

Redirect
Response

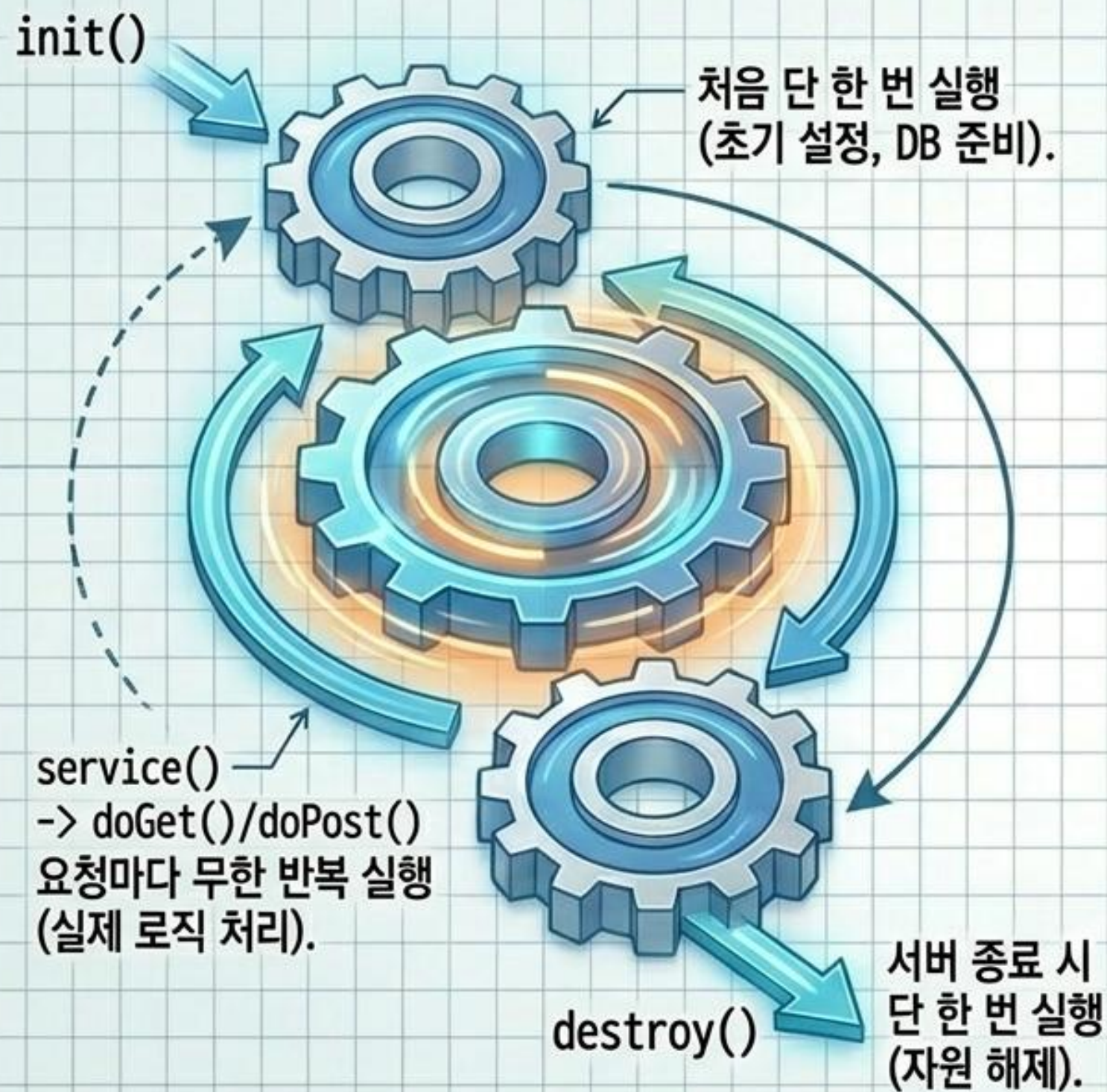
Request 2

JSP B

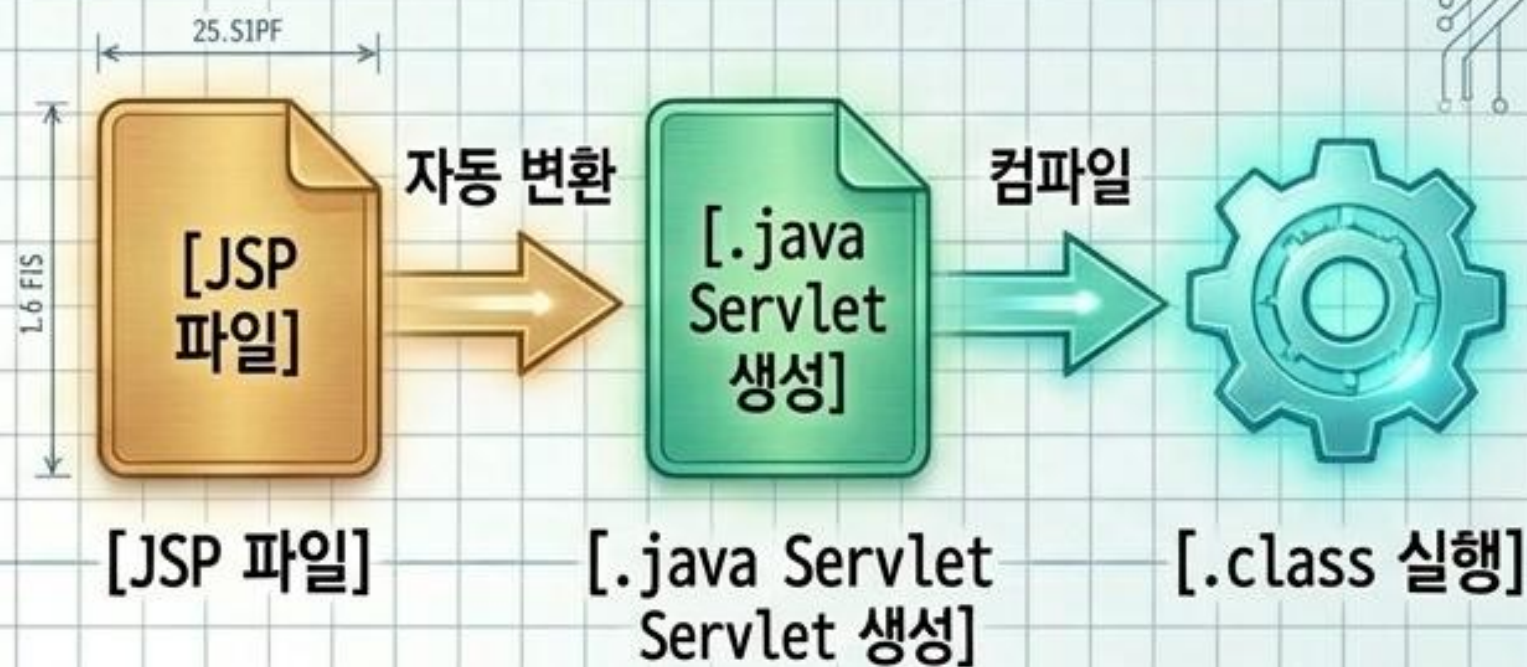
브라우저가 다시 요청. 속도 느림. request 데이터 소멸. URL 변경됨.

```
response.sendRedirect(url);
```

Servlet 생명주기 (Lifecycle)



JSP의 숨겨진 진실

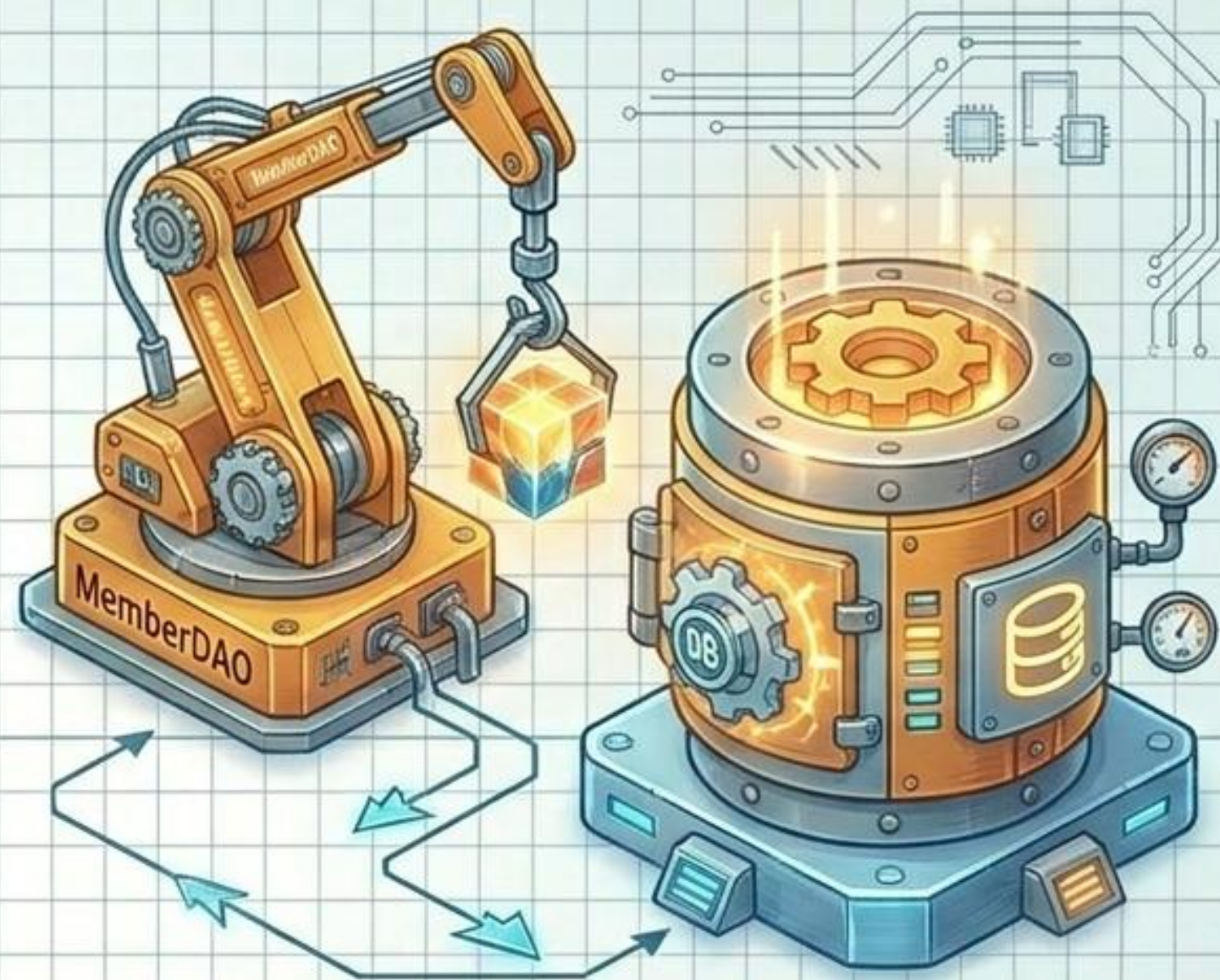


JSP는 HTML 중심의 뷰 기술이지만,
서버 내부에서는 결국 Servlet으로
자동 변환되어 실행된다.



DTO (Data Transfer Object)

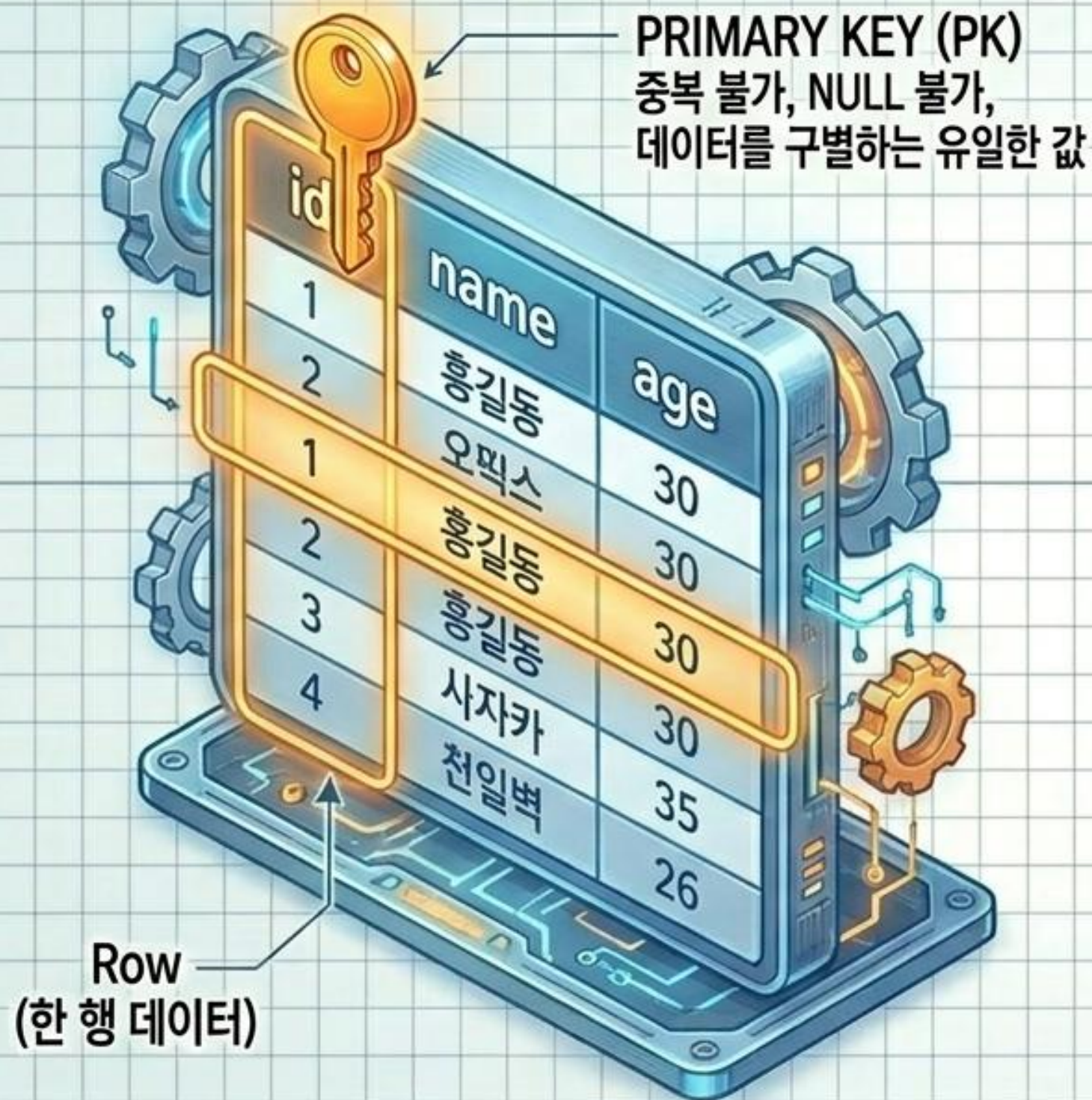
- 데이터 저장 및 운반 전용 객체
- 로직 없이 값(Value)만 담음
- 계층 간(Controller ↔ Model) 데이터 이동에 사용



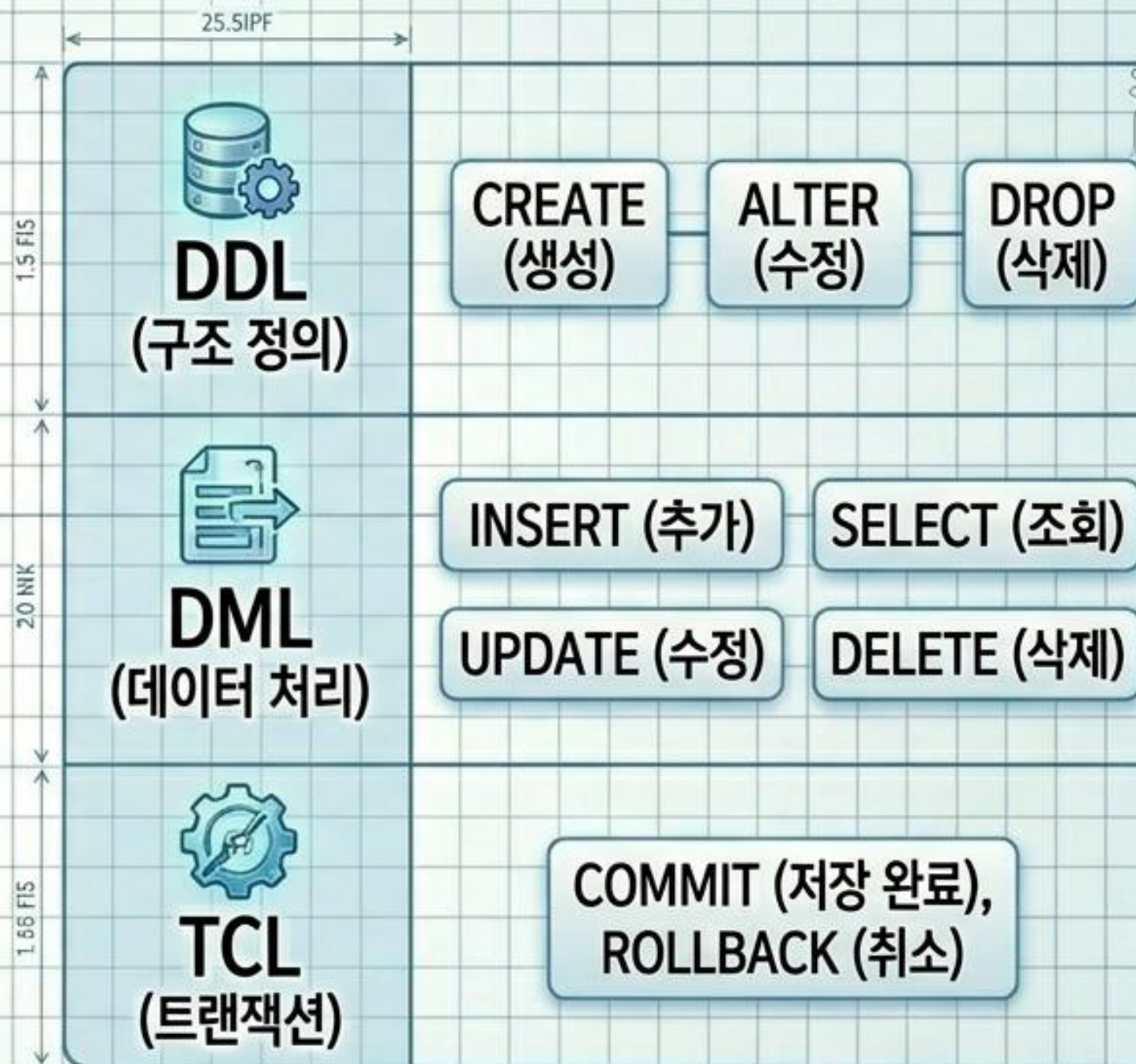
DAO (Data Access Object)

- 실제 DB 접근 및 연결 담당
- SQL 명령어 실행 전담
- 데이터 CRUD (생성, 조회, 수정, 삭제) 처리

MEMBER Table



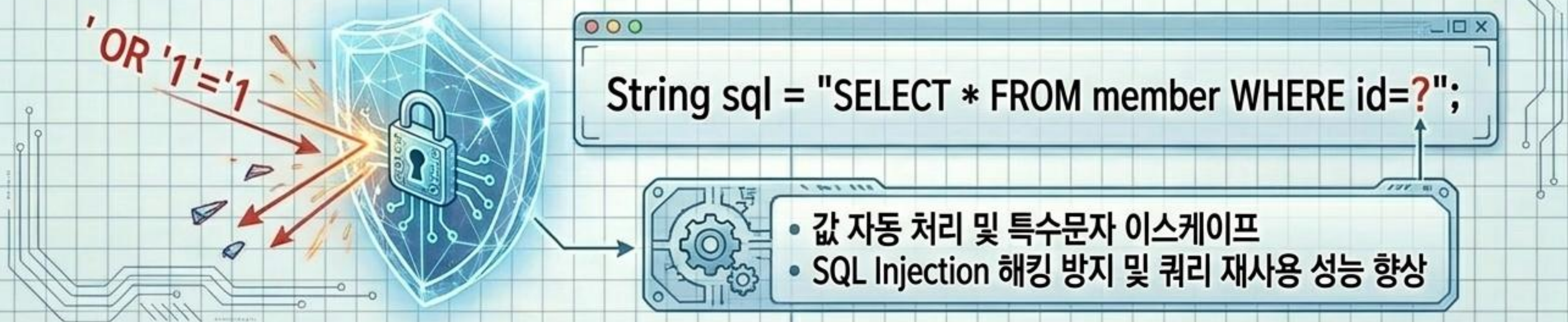
SQL 명령어 핵심 분류



Connection Pool: 성능 최적화



PreparedStatement: 보안과 성능의 방패



EL (Expression Language) 해부

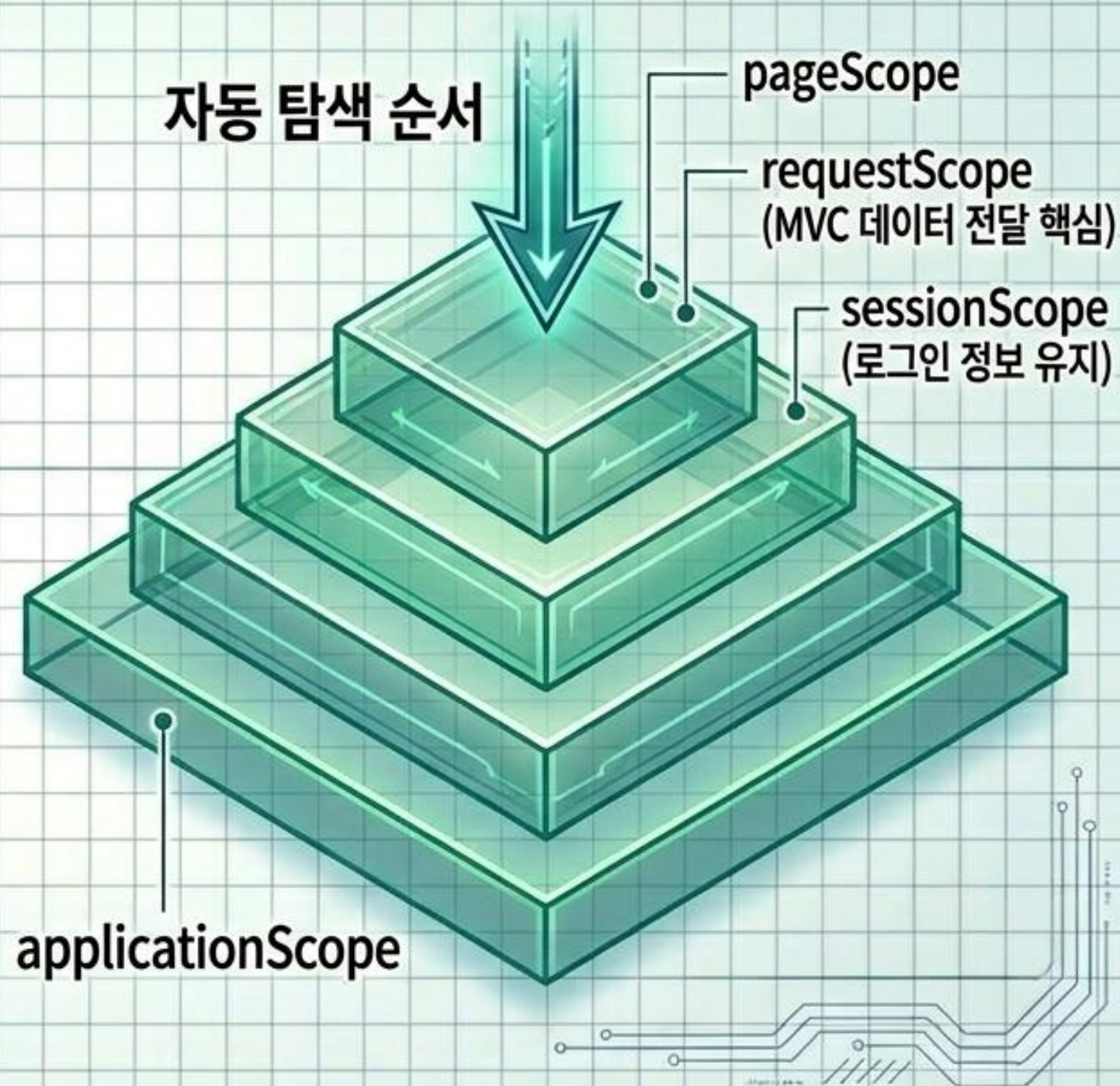
~~<%= request.getAttribute("name") %>~~

`${name}`

Java 객체에서 getName()을
자동으로 호출하여 JSP에 출력

 empty 연산자 : `${empty loginId}`
(값이 null이거나 비었을 때 true 반환)

EL의 자동 스코프(Scope) 탐색 과녁



JSTL: 자바 없는 깔끔한 뷰 컨트롤

<%@ taglib prefix="c" uri="..." %>

조건부 흐름 (c:if & c:choose)

```
<c:if test="${age} >= 20">  
  성인  
</c:if>
```

```
<c:choose>  
  <c:when test="...">A</when>  
  <c:otherwise>F</otherwise>  
</c:choose>
```

데이터 반복 (c:forEach)

```
<c:forEach var="m"  
  items="${list}">  
  ${m.name}  
</c:forEach>
```

varStatus 속성으로
index, count 상태
관리 가능

데이터 포맷 및 함수 (fmt & fn)

```
<fmt:formatDate  
  value="${date}"  
  pattern="yyyy-MM-dd"/>  
<fmt:formatNumber  
  value="1000"  
  pattern="#,###"/>
```

문자열 처리:
\${fn:length(str)}
\${fn:contains(str, 'Java')}

Controller (Servlet.java)

```
1  
2  
3  
4  
5  
6  
7  
List<Member> list = dao.getMembers();  
request.setAttribute("list", list);
```

DB에서 가져온 데이터를 request 객체에 'list'라는 이름으로 포장하여 저장

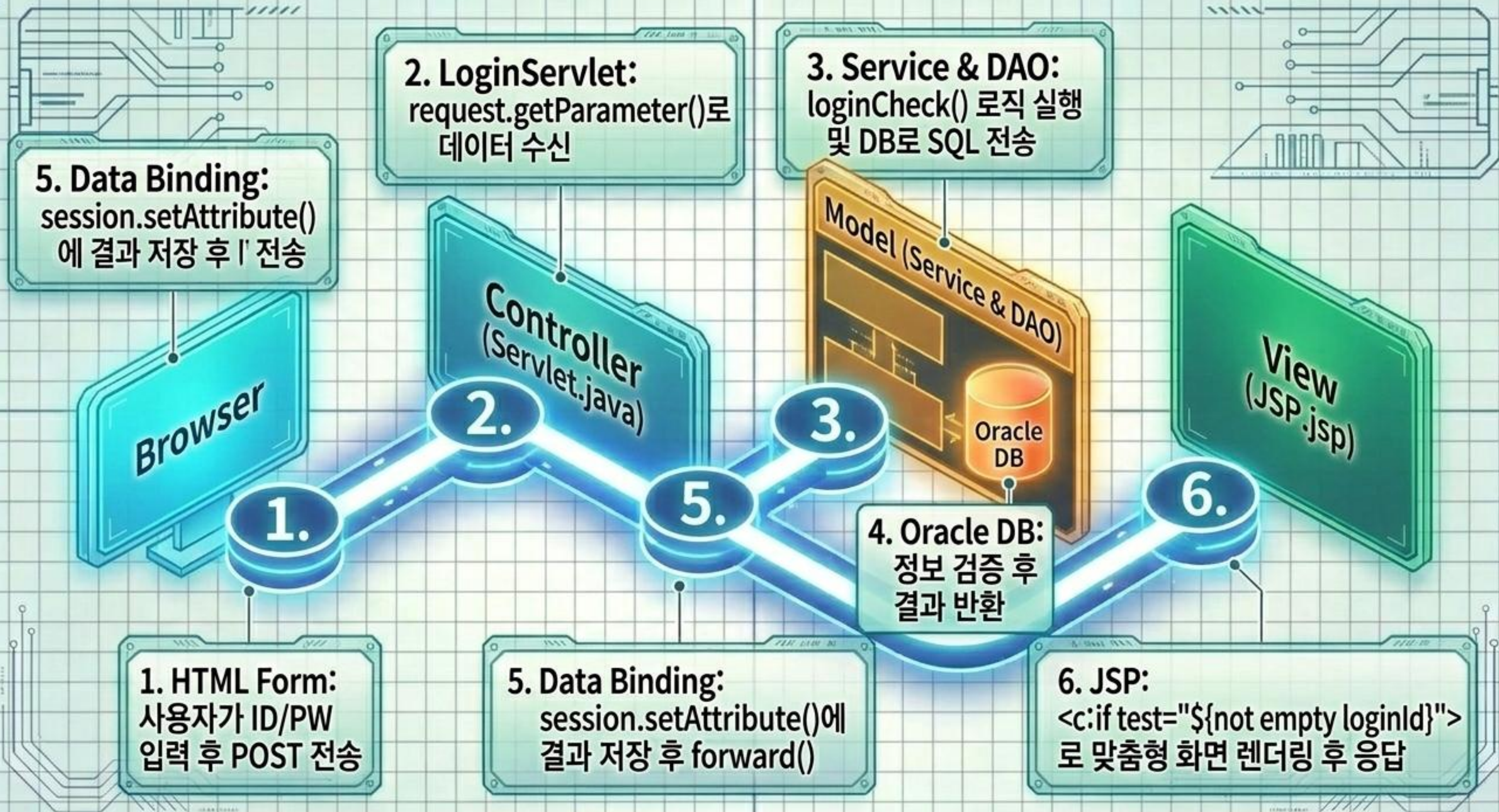
forward() 이동

list

View (JSP.jsp)

```
1  
2  
3  
4  
5  
6  
7  
<c:forEach var="m" items="${list}">  
  <li>${m.name}</li>  
</c:forEach>
```

EL과 JSTL이 request 객체에서 'list'를 자동으로 찾아내어 HTML로 반복 렌더링



시험 직전 & 실무 투입 전: 핵심 요약 치트시트

아키텍처 역할 분담

- Servlet: Controller (요청 수신, 흐름 제어, Java 중심)
- Java/DB: Model (비즈니스 로직, JDBC 연결, 데이터 처리)
- JSP: View (화면 렌더링, HTML 중심)



Servlet 핵심 동작

- 생명주기: `init()` -> `service()` -> `doGet/doPost()` -> `destroy()`
- forward: 서버 내 이동, request 유지, URL 고정
- redirect: 브라우저 재요청, request 소멸, URL 변경



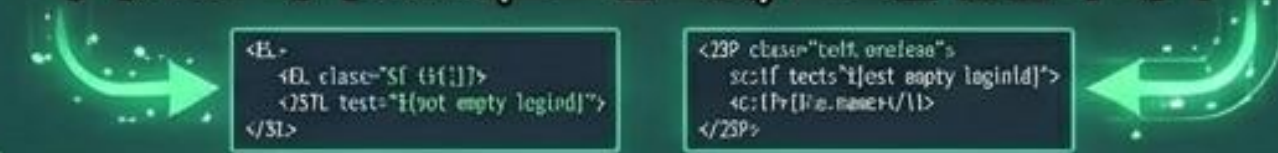
DB & JDBC 필수 개념

- Connection Pool: DB 연결 미리 생성 및 재사용 (속도 향상)
- PreparedStatement: 쿼리 재사용 및 SQL Injection 방지
- DAO (접근 수행) / DTO (데이터 운반)

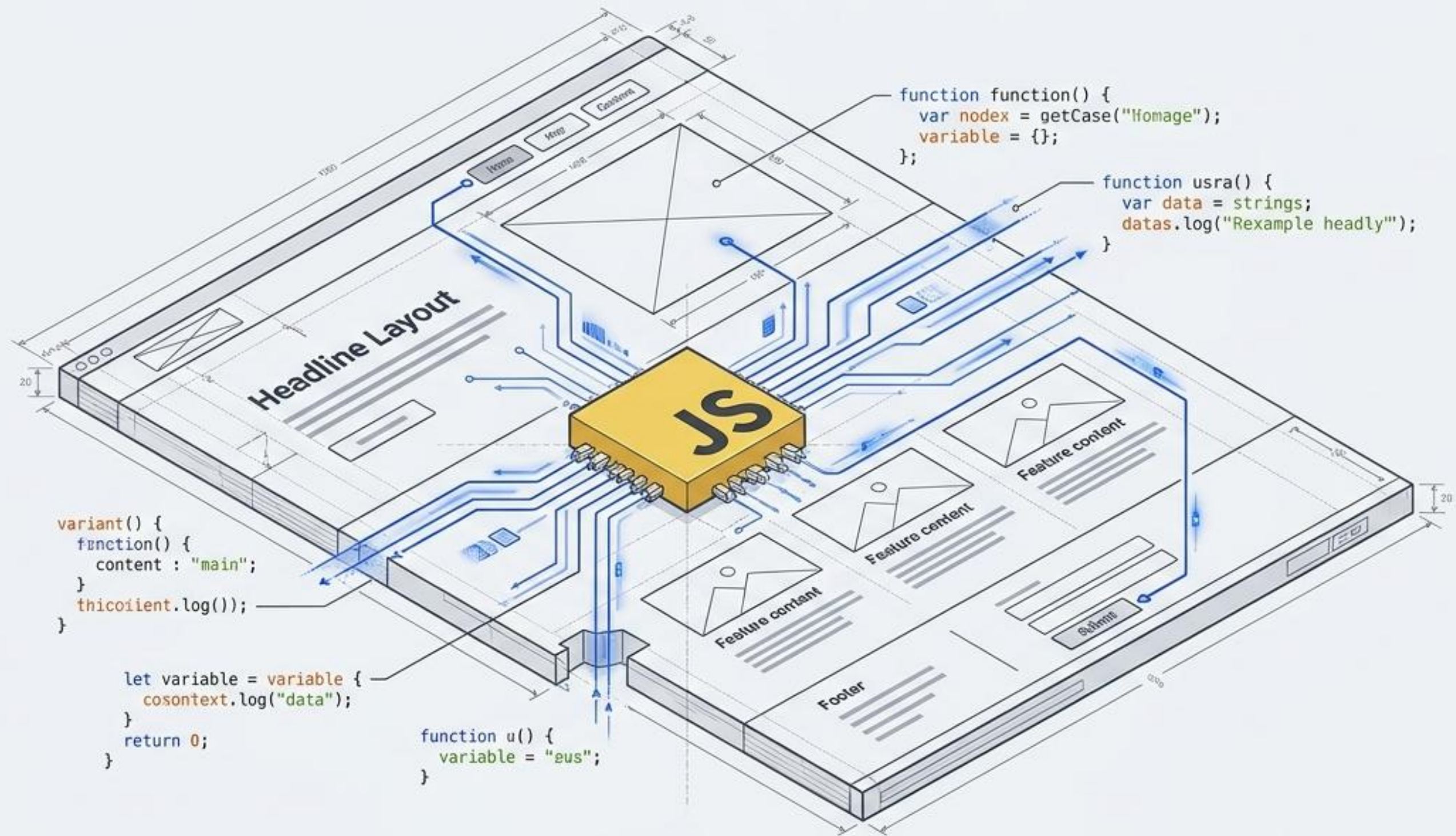


JSP 핵심 태그

- EL (`${}`): 데이터 자동 스코프 탐색 및 간결한 출력
- JSTL core: `<c:forEach>` (데이터 반복), `<c:if>` (조건부 흐름)
- 목적: JSP에서 Java(스크립트릿) 코드를 완전히 제거

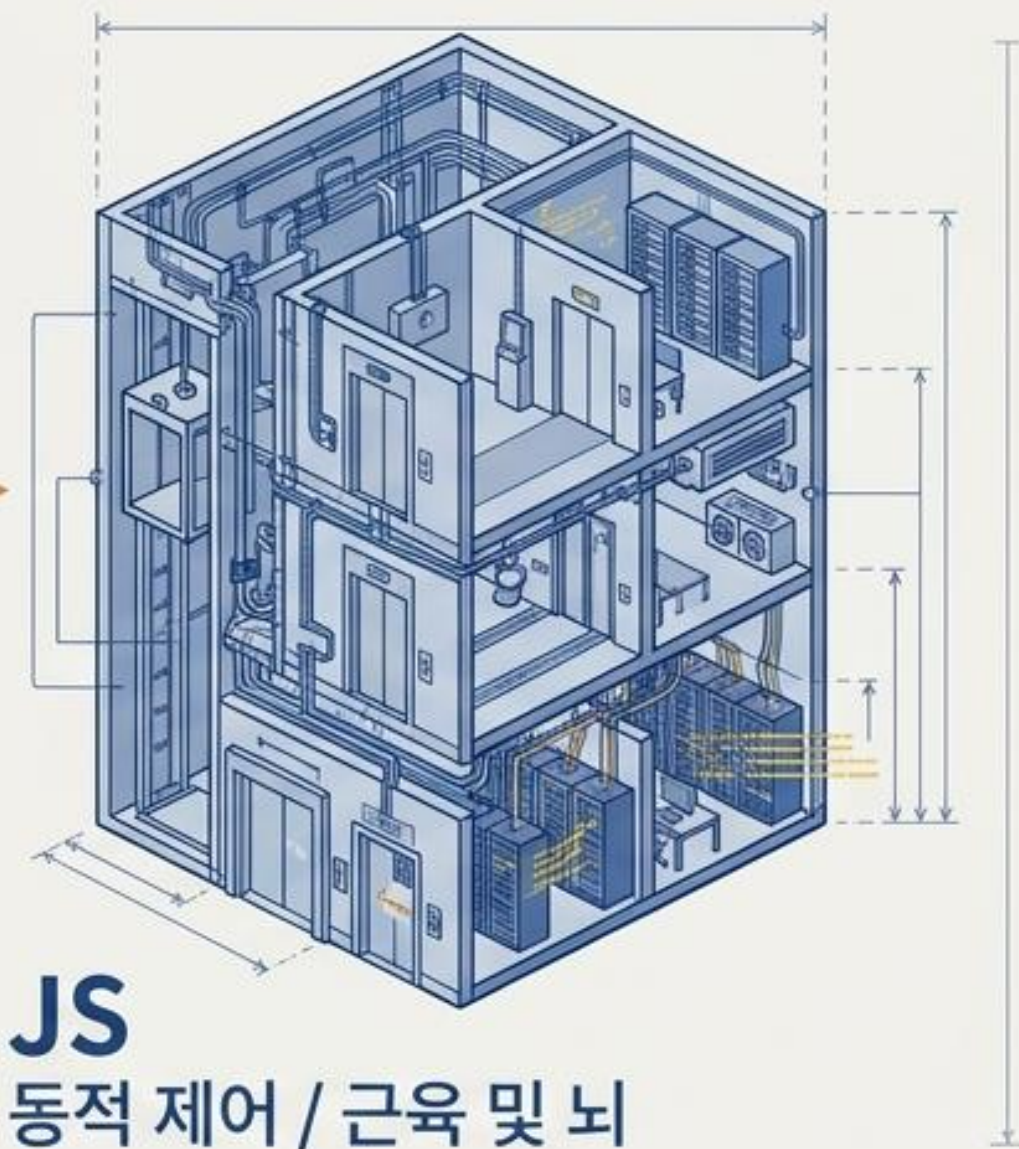
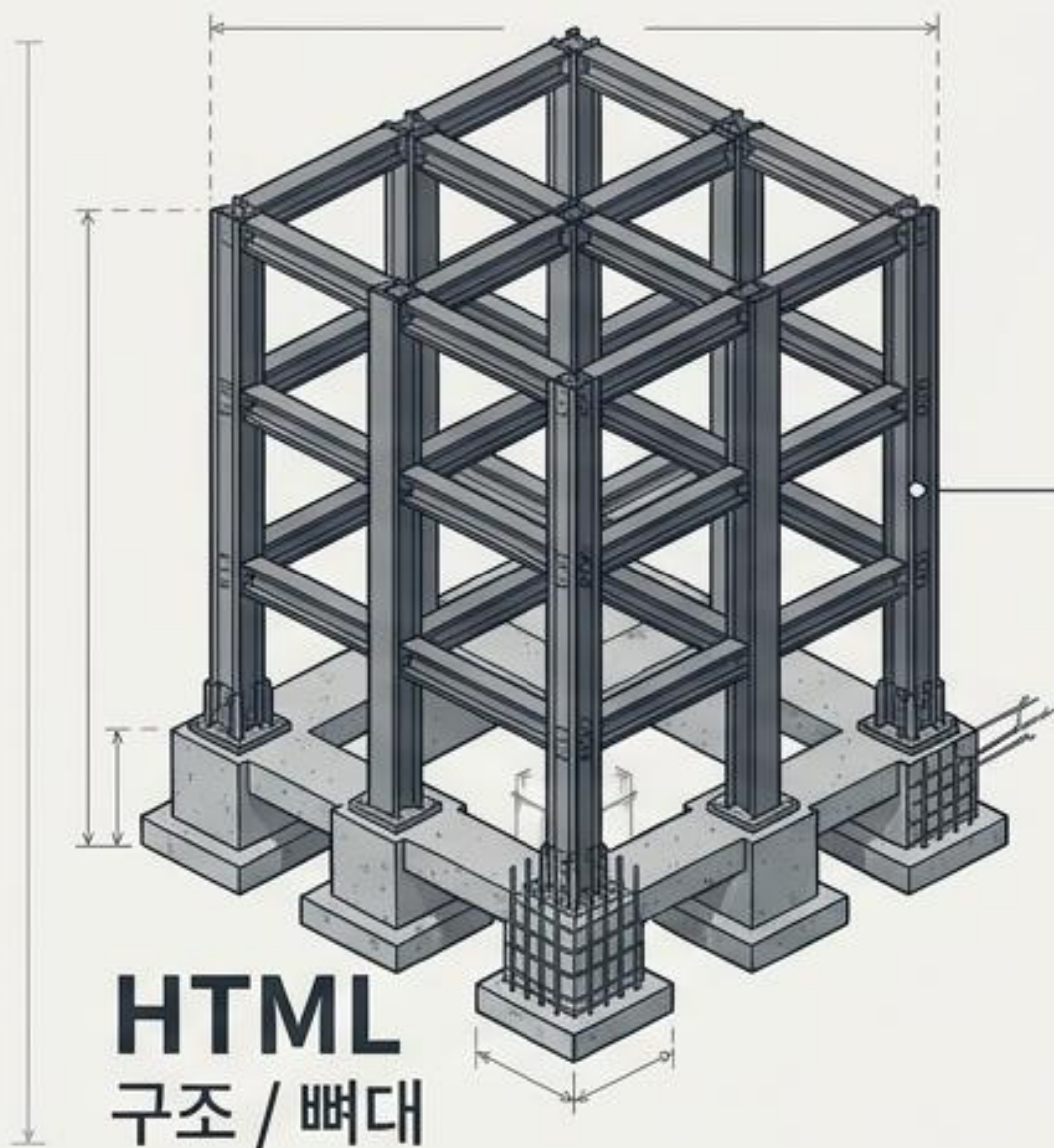


JSP는 화면(View), Servlet은 제어(Controller), Java 클래스는 처리(Model)를 담당하며 MVC 구조를 통해 유지보수성과 확장성을 극대화한다.



JavaScript 기초 완전 정리

데이터부터 DOM 제어까지, 웹을 움직이는 동적 언어의 핵심 바이블.



JavaScript란?

웹 페이지를 동적으로 움직이게 만드는 프로그래밍 언어.

인터프리터 언어

동적 타입

객체 기반

이벤트 중심

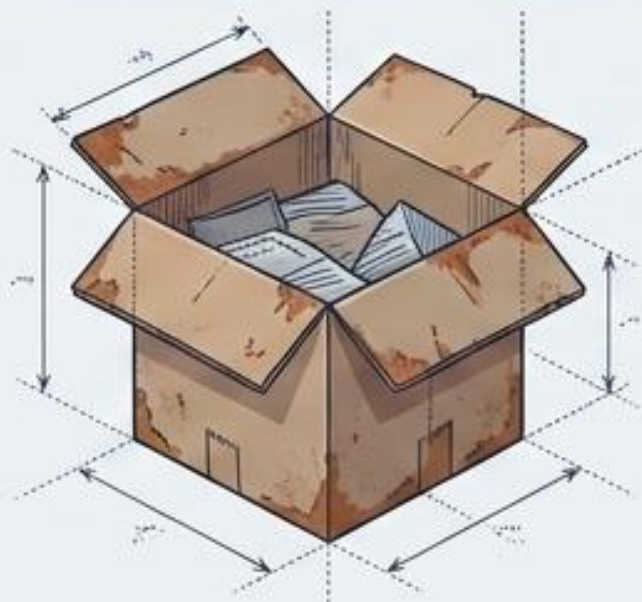
주요 역할

이벤트 처리	버튼 클릭 및 사용자 상호작용
자동 계산	데이터 연산 및 결과 도출
화면 제어	HTML/CSS 구조 및 색상 실시간 변경
데이터 구조화	배열과 객체를 활용한 정보 관리
서버 통신	AJAX를 통한 비동기 데이터 교환

변수 (Variables): 메모리 저장소

1개의 데이터를 저장하는 독립적인 메모리 공간.

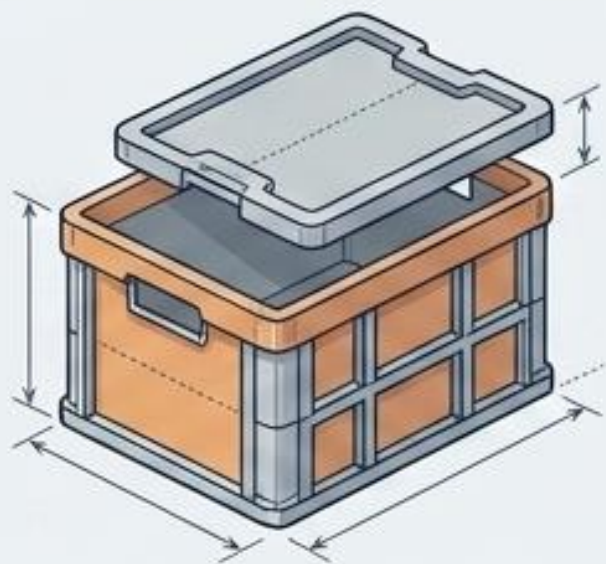
var



- ES5 (옛날 방식)
- 스코프 불명확
- 재선언, 재할당 가능
- 사용 비권장

```
var name = 'Kim';
```

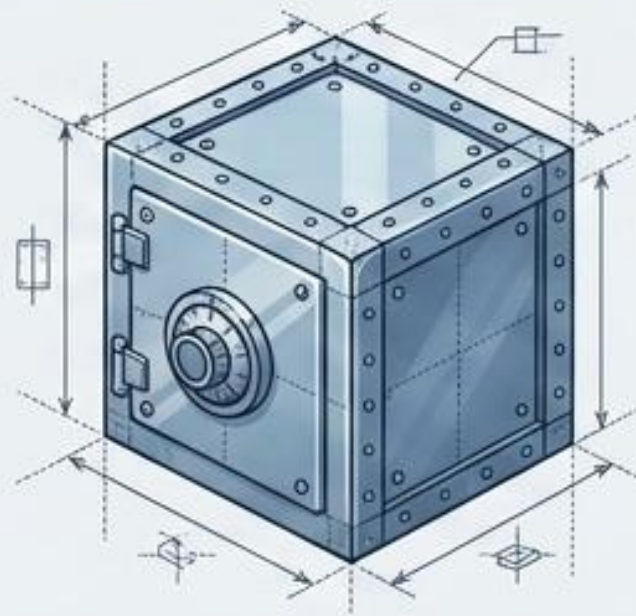
let



- ES6 (현재 표준)
- 블록 스코프 (지역 변수)
- 재할당 O, 재선언 X
- 값이 변하는 데이터에 사용

```
let age = 20;
```

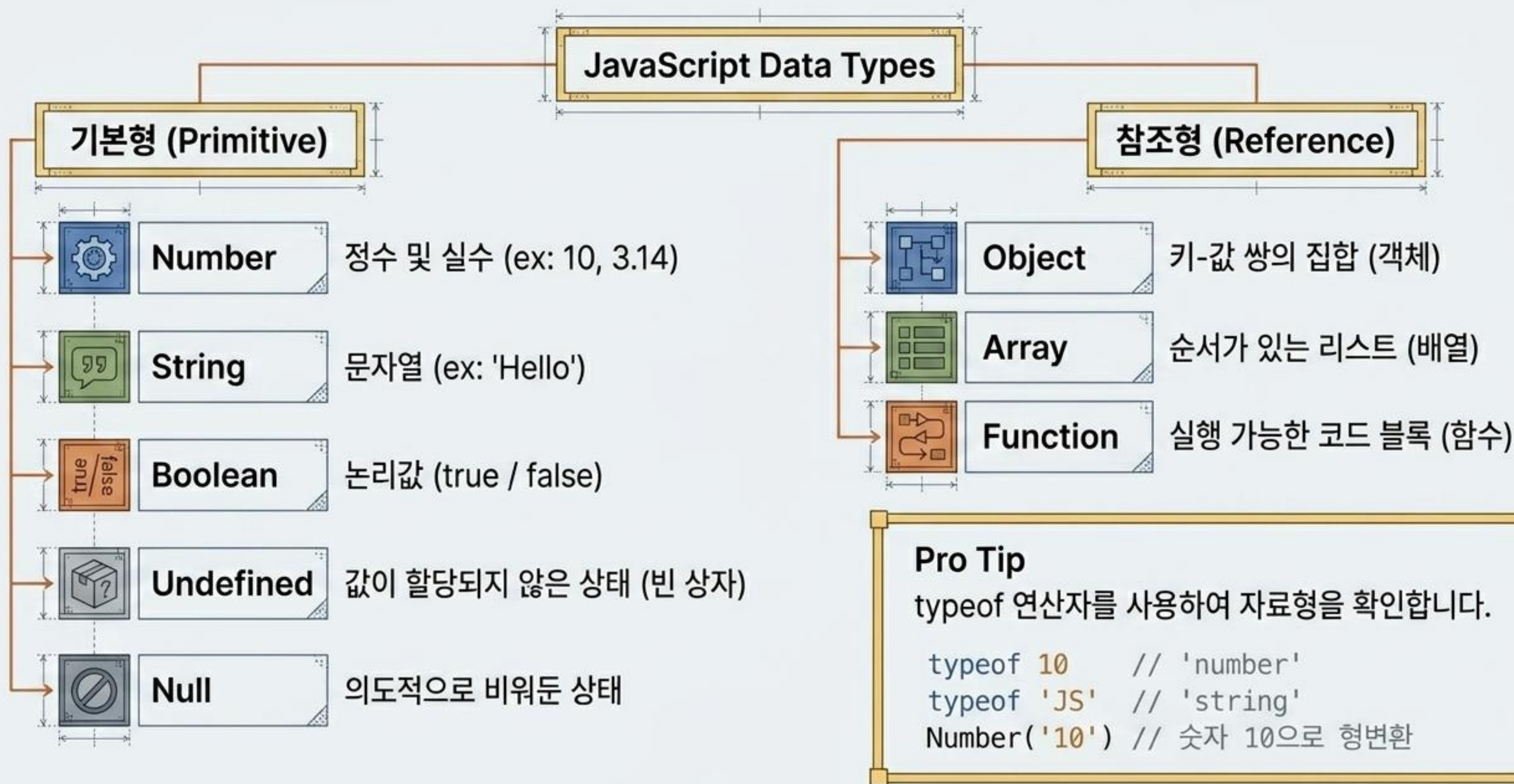
const



- ES6 상수형 변수
- 블록 스코프
- 재할당 X, 재선언 X
- 안전한 데이터 보존에 사용

```
const pi = 3.14;
```

데이터형 (Data Types)





출력 방식 (Output Destinations)

1



```
console.log('Hello');
```

개발자 도구 콘솔에 출력.
실무에서 디버깅 용도로 가장 많이 사용됨.

2



```
document.write('Hello');
```

브라우저 HTML 화면에 직접 출력.
(현재는 DOM 조작으로 대체됨).

3



```
alert('Hello');
```

사용자에게 시스템 경고 및 팝업창으로 알림.

실행 위치 (Execution)



내부: `<script>` 태그 안



외부: `<script src='main.js'>` (실무 권장)

연산자 (Operators): 데이터 조작 패널

산술 및 증감 (Arithmetic)



+
(더하기)



-
(빼기)



*
(곱하기)



/
(나누기)



%
(나머지)

숫자 0으로
나누면
Infinity
반환



++
(1 증가)



--
(1 감소)

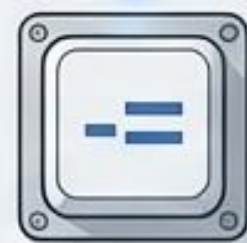
대입 (Assignment)



오른쪽 값을
왼쪽에 대입



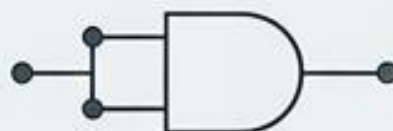
기존 값에
더해서 대입



기존 값에서
빼서 대입

논리 (Logical)

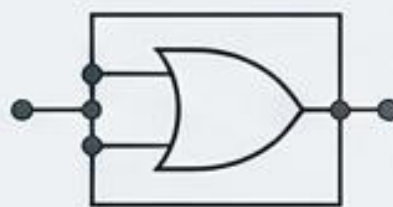
&&
(AND)



	T	F
T	T	F
F	F	F

직렬 조건 (양쪽 모두 true일 때만 true)

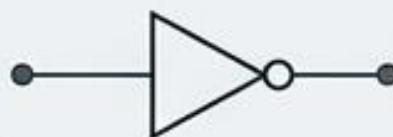
||
(OR)



	T	F
T	T	T
F	T	F

병렬 조건 (하나라도 true이면 true)

!
(NOT)



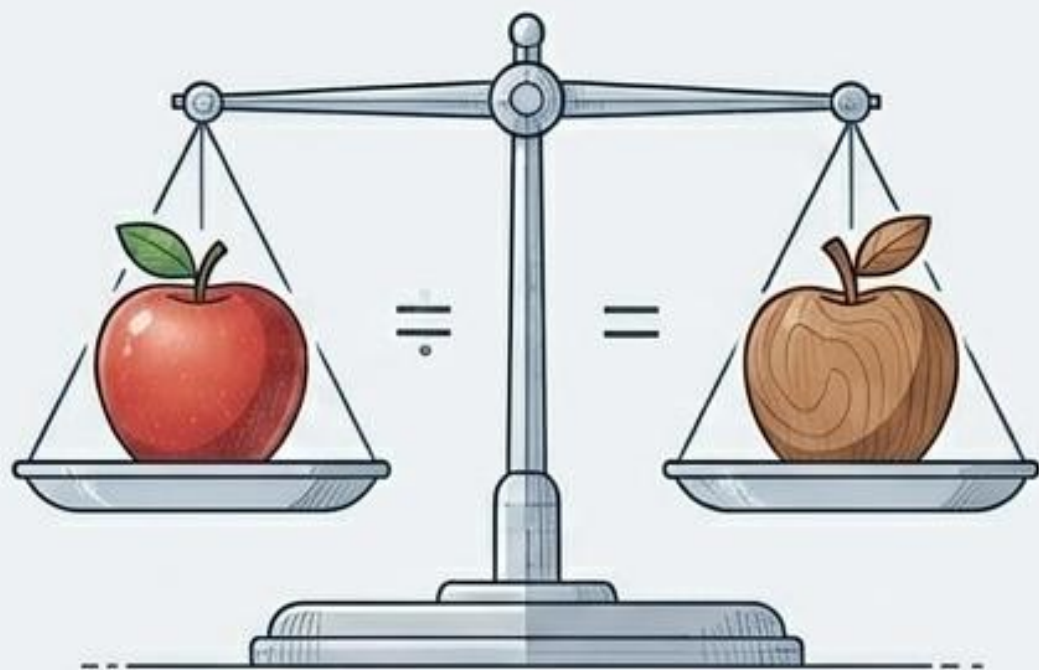
	T	F
T	F	F
F	T	T

상태 반전 (true -> false)

비교 연산의 핵심: `==` vs `===`

느슨한 비교 (Loose Equality)

`==`

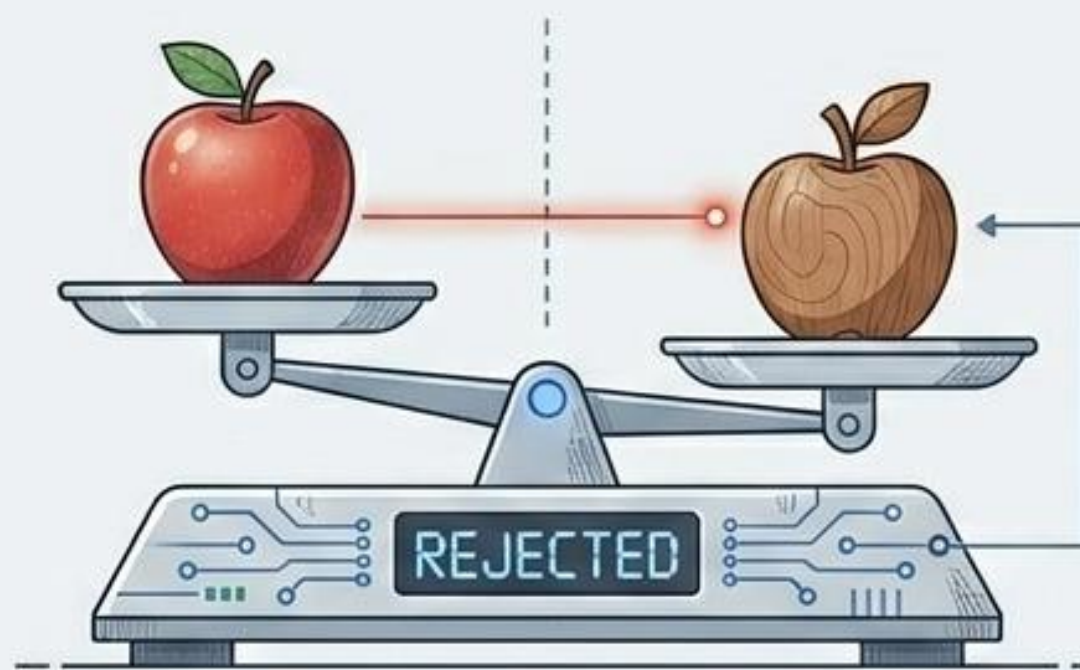


데이터형을 무시하고 '값'만 같으면 true를 반환합니다.

`10 == '10' → True` (숫자와 문자열을 강제로 맞춰서 비교)

엄격한 비교 (Strict Equality)

`===`



값과 '자료형'이 모두 완벽히 일치해야만 true를 반환합니다.

`10 === '10' → False` (데이터형이 다르므로 거절)

실무 가이드: 예기치 않은 데이터 타입 오류를 방지하기 위해 반드시 `===` 와 `!==` 사용을 권장합니다.

제어문: 조건에 따른 분기 (Control Flow)

if / else if / else

범위나 논리 조건이 필요할 때 사용 (유효성 검사 등).

```
if (score >= 90) {  
    return 'A';  
} else if (score >= 80) {  
    return 'B';  
} else {  
    return 'F';  
}
```

switch

다중 특정 값을 정확히 일치시켜 비교할 때 사용. (각 case마다 break 필수).

```
switch(num) {  
    case 1: break;  
    default: break;  
}
```

삼항 연산자 (Ternary)

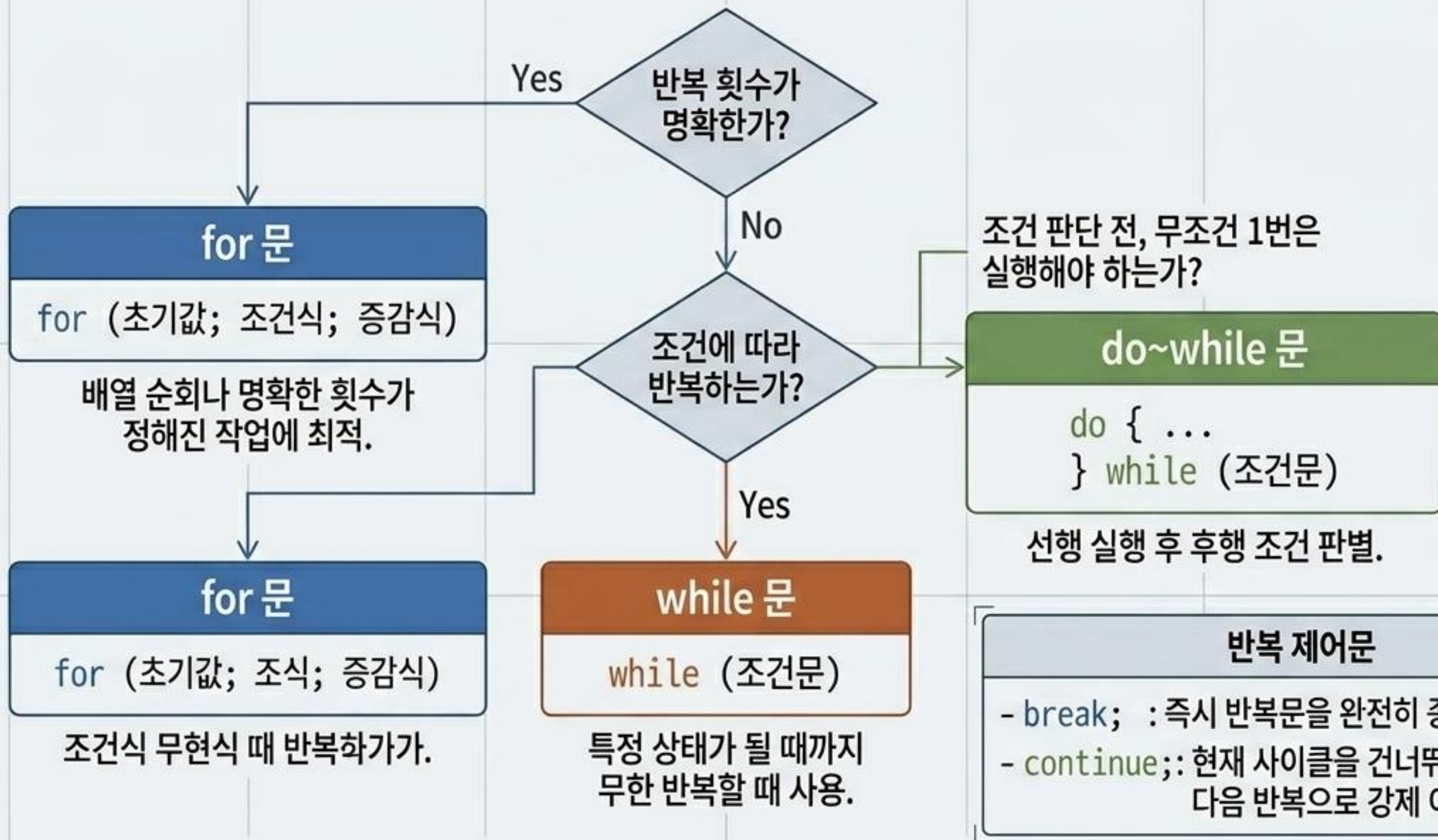
if~else를 한 줄로 압축한 숏컷 문법.

조건 ? 참일 때 값 : 거짓일 때 값;

switch

다중 특정 값을 정확히 일치시켜 비교할 때 사용. (각 case마다 break 필수).

어떤 반복문을 사용해야 할까? (Loop Decision Tree)

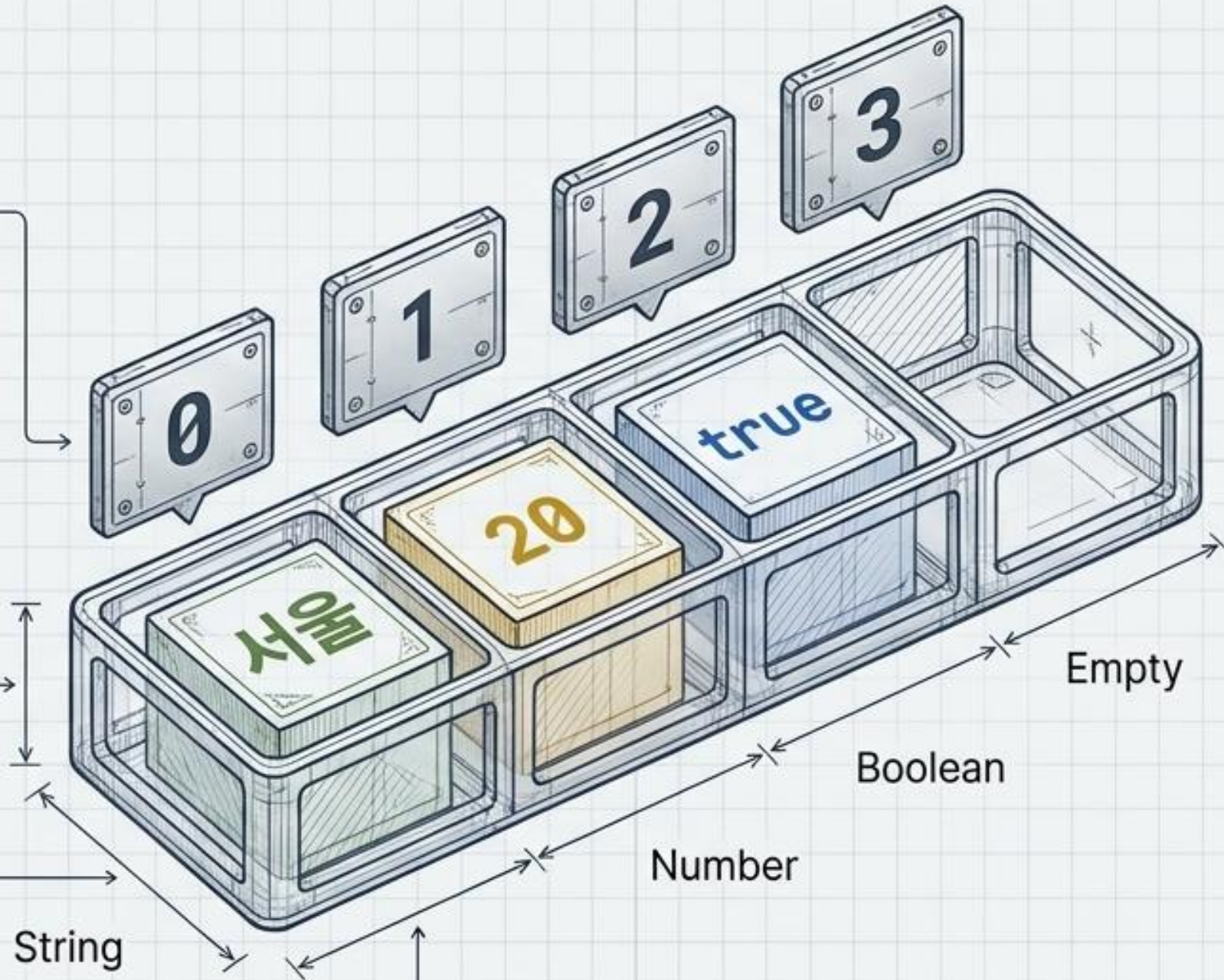


배열 (Arrays): 순서 기반 데이터 구조

특징: 여러 데이터를
순서대로 저장하는 자료구조.
JS 배열은 다른 데이터형을
혼합하여 저장 가능합니다.

접근: `arr[인덱스]`
(0부터 시작)

길이: `arr.length`
(데이터의 총 갯수 반환)



```
let arr = ["서울", 20, true];
```

주요 조작 메서드 (Mutations)



`push()`

배열의 맨 마지막에
새로운 데이터 추가.



`pop()`

배열의 맨 마지막
데이터 삭제.



`splice(index, count)`

원하는 위치의 데이터를
삭제하거나 새 데이터를
중간에 삽입.

자료구조 비교: Array vs Object

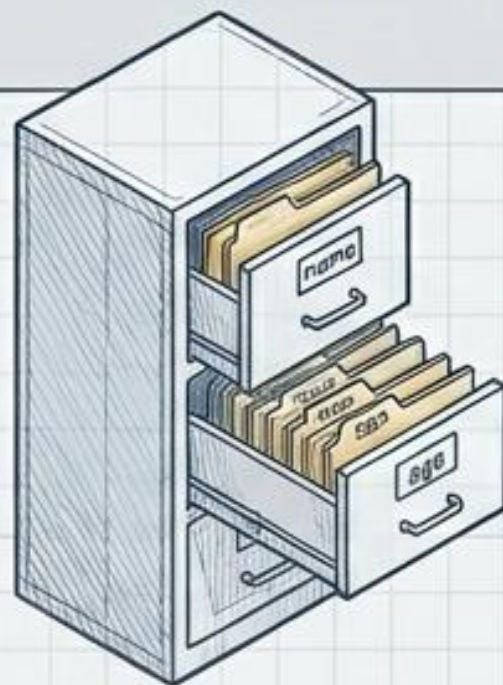
배열 (Array)



구조	[값, 값, 값]
저장 방식	순서 (Index) 기반
데이터 접근	arr[0]

```
let arr = ['Kim', 20];
```

객체 (Object)



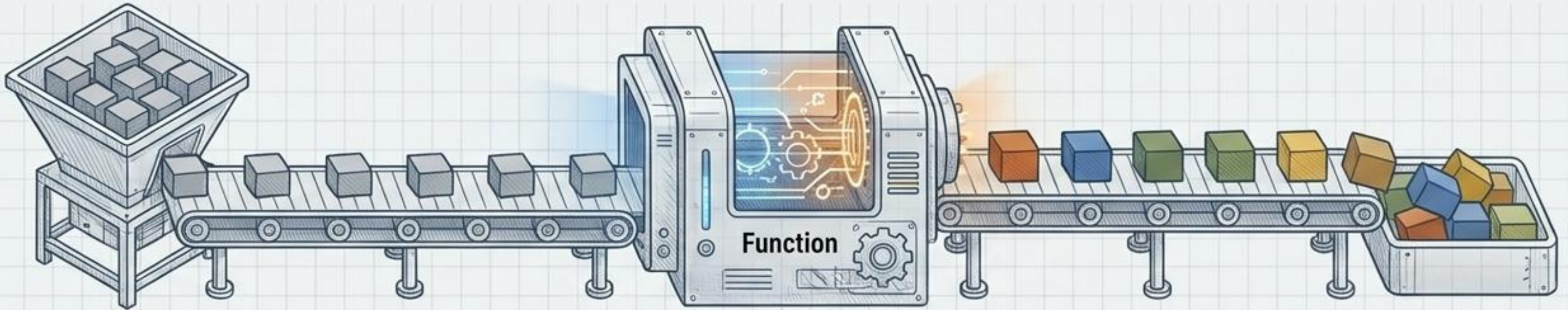
구조	{키: 값, 값} (JSON 형태)
저장 방식	이름 (Key) 기반
데이터 접근	member.name

```
let member = { name: 'Kim', age: 20 };
```

실무 활용: 배열 안에 객체가 담긴 형태 (Array of Objects)로 서버 데이터(API)를 가장 많이 처리합니다.

```
const movies = [ {id: 1, title: 'Dune'}, {id: 2, title: 'Tenet'} ];
```

모던 배열 반복문 (Modern Array Iteration)



for...of

배열의 '실제 데이터 값'을 1개씩 순차적으로 읽어옴.

```
for (let item of arr) { ... }
```

forEach() (권장)

실제 데이터를 1개씩 읽어 콜백 함수를 실행함. 단순 반복 목적.

```
arr.forEach(item => { console.log(item); });
```

for...in

배열의 '인덱스' 또는 객체의 '키(Key)'를 읽어옴.

```
for (let index in arr) { ... }
```

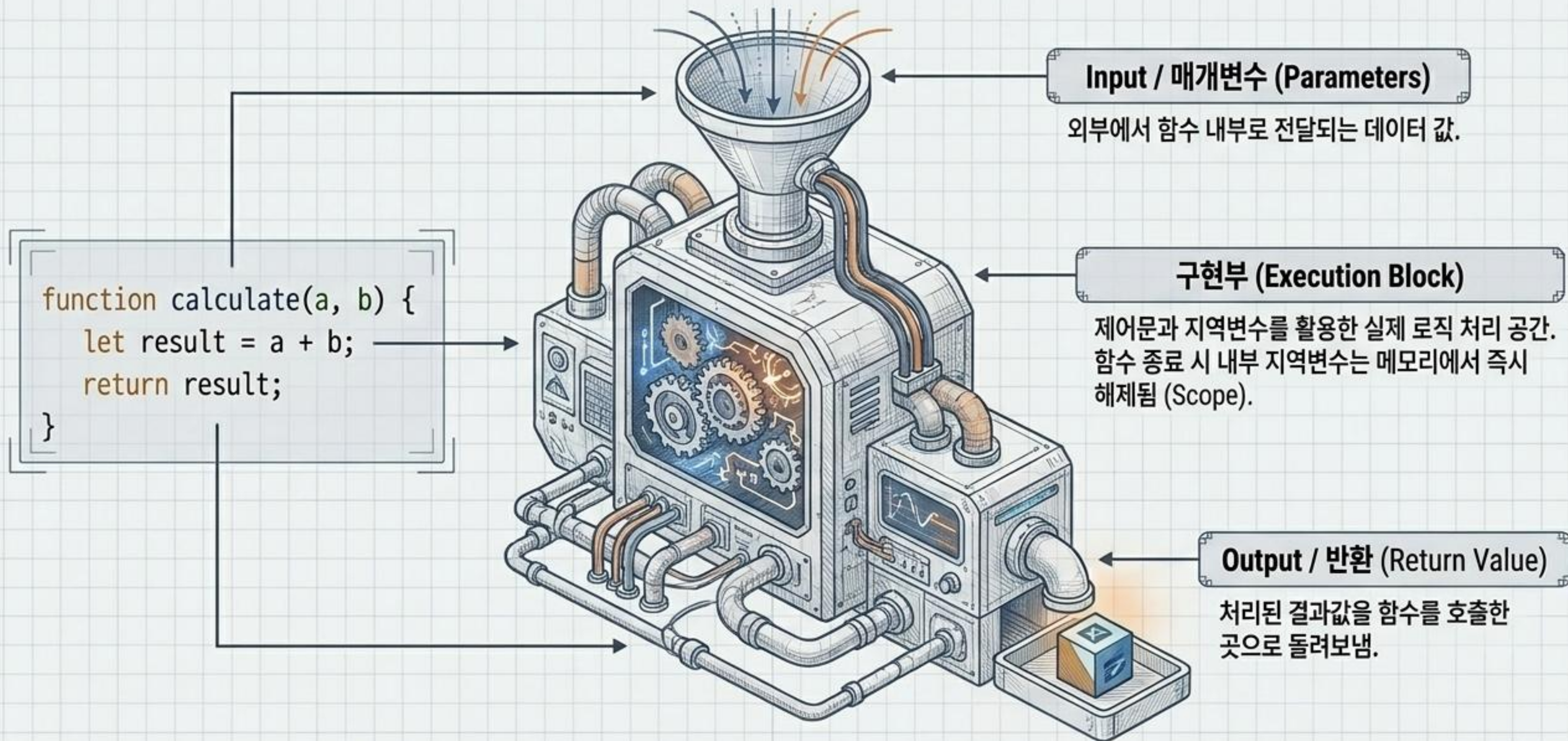
map() (가장 중요)

데이터를 1개씩 읽어 연산한 후, 완전히 새로운 배열을 반환함. (React 핵심).

```
let newArr = arr.map(item => item * 2);
```

함수 (Functions): 로직 팩토리

명령문의 집합이자 구조화 프로그램의 단위. 반복을 제거하고 재사용성을 높입니다.



함수 문법의 진화 (ES5 → ES6)

선언적 함수 (Standard Function)

전통적인 방식.

```
function plus(a, b) {  
  return a + b;  
}
```

익명 함수 (Anonymous Function)

함수를 데이터처럼 변수에 할당하는 방식.

```
let plus = function(a, b) {  
  return a + b;  
}
```

화살표 함수 (Arrow Function)

가장 현대적인 방식. function 키워드와 return을 생략하여 극도로 압축.

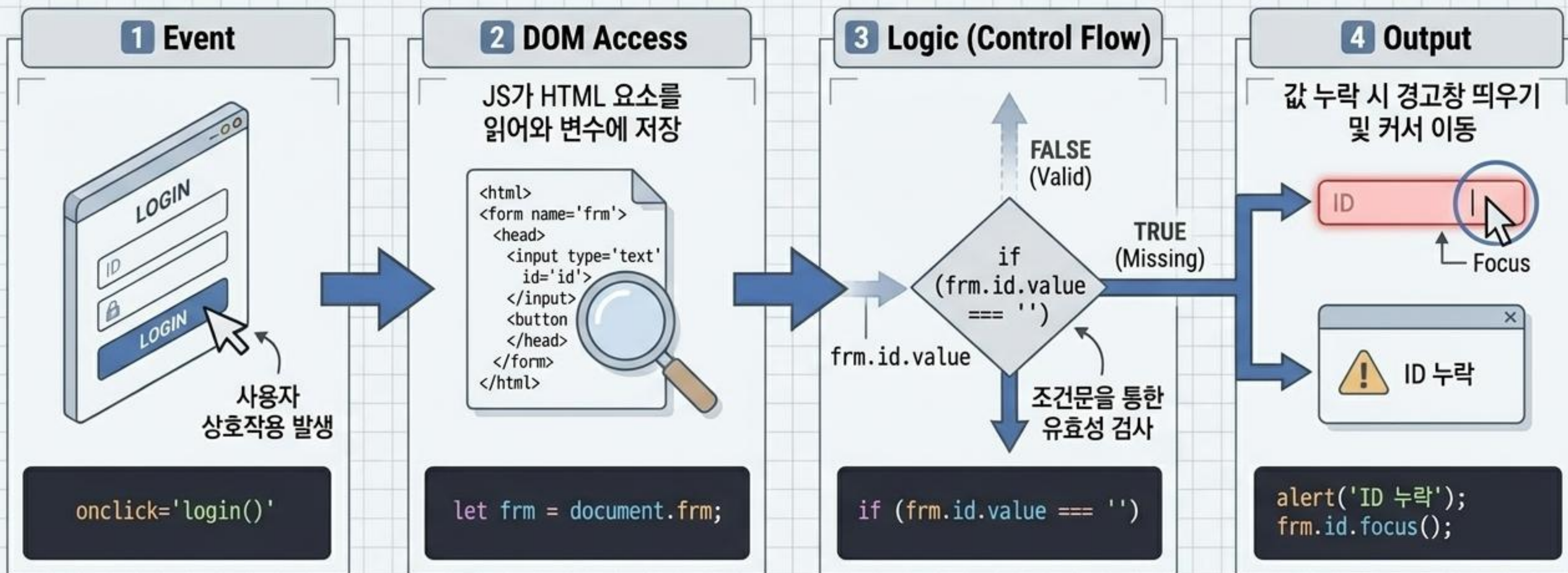
```
let plus = (a, b) => a + b;
```

콜백 (Callback) 함수 패턴

함수 자체를 다른 함수의 매개변수(Parameter)로 전달하여, 시스템 로직이나 특정 이벤트가 발생했을 때 내부적으로 자동 호출되게 하는 핵심 설계 패턴입니다.

```
setTimeout(() => {  
  console.log('3초 후 실행');  
}, 3000);
```

Synthesis: DOM과 이벤트의 만남 (로그인 폼 제어)



최종 요약: JavaScript는 변수, 연산자, 제어문, 함수를 조합하여 정적인 HTML 문서를 사용자와 상호작용하는 동적인 애플리케이션으로 변화시킵니다.